

가상현실 REPORT



과목 이름: 가 상 현 실

분반: 0 1 분 반

교수님 성함: 박 원 철 교 수 님

학과: 컴 퓨 터 공 학 부

전공: 소 프 트 웨 어 전 공

학번: 2 0 1 9 0 1 7 6 9

이름: 김 완 규

제출일자: 2 0 2 4 . 0 6 . 1 8 . (화)

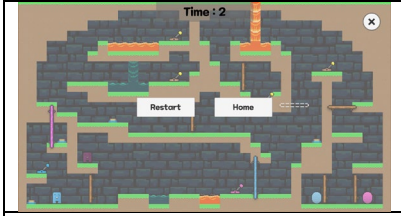
- **게임 이름 :** Two Blobs Escape Game.
- **게임 장르 :** 1인용, 2D 플랫폼머, 퍼즐, 미로, 탈출.

- **게임 설명 및 목표 :**
 두 플레이어를 조작하여, 장애물은 피하고, 퍼즐을 풀면서, 서로를 도와서
 두 플레이어가 모두 목표 지점까지 도달하는 게임입니다.

- **조작 방법 :**

플레이어 이동	←, →
플레이어 변경(스왑)	C
플레이어 점프	Space
플레이어 상호작용(스위치)	F
UI 버튼 및 퍼즐 조작	마우스 좌클릭

- **게임 화면 :**

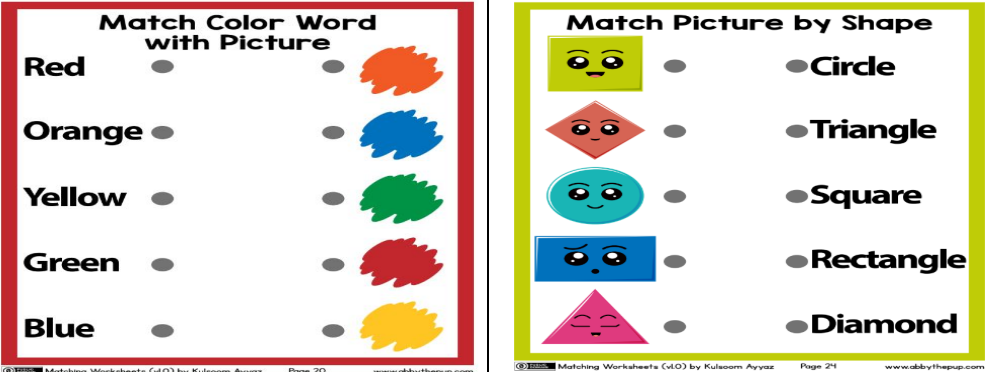
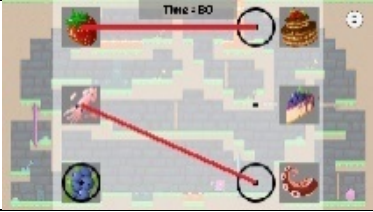
		
시작화면	게임화면	퍼즐
		
옵션	게임 클리어	게임 오버

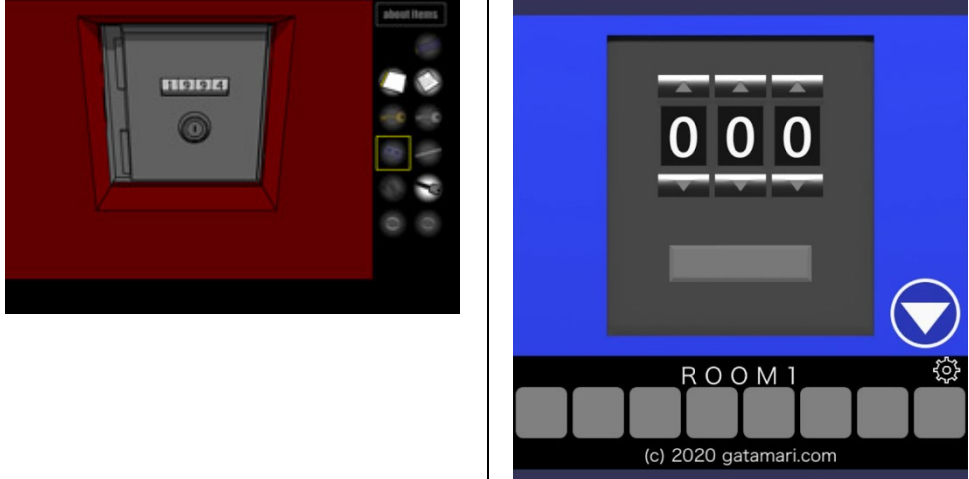
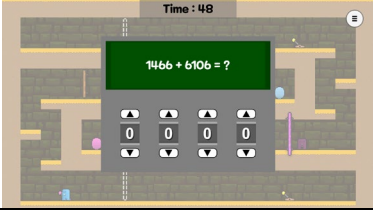
● 주요 참고 게임 :

Fireboy & Watergirl, 직소 퍼즐 및 연결 게임, 방탈출 게임.

Fireboy & Watergirl 시리즈 (2인용 협동 퍼즐 플랫폼 게임)		
참고 게임	Fireboy & Watergirl: Forest	Fireboy & Watergirl: Elements
예시 이미지 출처	https://play.google.com/store/apps/details?id=com.osloalbet.forest&hl=en_US	https://store.steampowered.com/app/1003480/Fireboy_Watergirl_Elements/
게임 내 구현 이미지		

직소 퍼즐 (드래그 앤 드롭 형식)		
참고 게임	Puzzle Together Multiplayer Jigsaw Puzzles	직소 퍼즐 - 퍼즐 게임
예시 이미지 출처	https://store.steampowered.com/app/1478220/Puzzle_Together_Multiplayer_Jigsaw_Puzzles/?l=koreana	https://play.google.com/store/apps/details?id=com.easybrain.jigsaw.puzzles&hl=ko
게임 내 구현 이미지		

Matching (알맞은 것끼리 연결하는 퍼즐 게임)	
참고 게임	Matching Worksheets (36p) - Abby the Pup
예시 이미지 출처	(https://www.abbythepup.com/) https://drive.google.com/file/d/1Etu5no2ymuFYuPueiefFa0SzJ4314K60/view
게임 내 구현 이미지	

비밀번호 퍼즐 (비밀번호를 맞 추어 금고나 문 을 여는 퍼즐)	
참고 게임	Crimson Room 시리즈 中 Crimson Room(플래시 게임) PAS5W0RDS
예시 이미지 출처	https://www.b-sting.nl/crimsonroom/solve.html https://play.google.com/store/apps/details?id=com.gatamari.PAS5W0RDS&hl=ko&gl=US
게임 내 구현 이미지	

● 주요 구성 요소 :

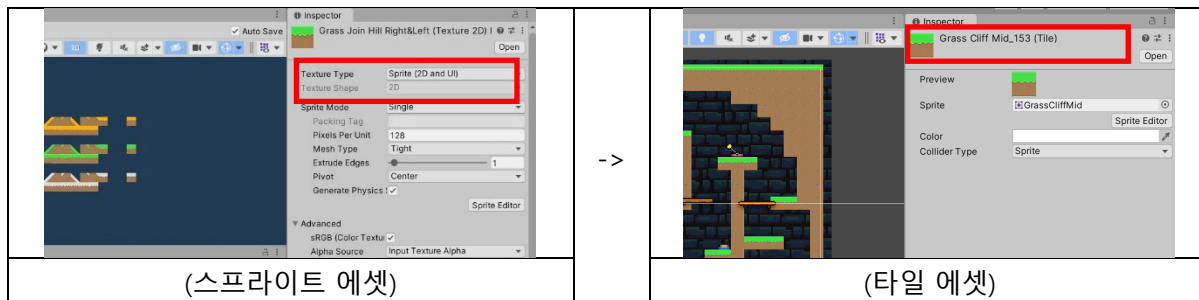
지형, 플레이어, 게임매니저(+키&Sfx), 오브젝트(버튼, 스위치, 문, 플랫폼), 목표지점, 물&용암, UI, 퍼즐

● 구현 방법 :

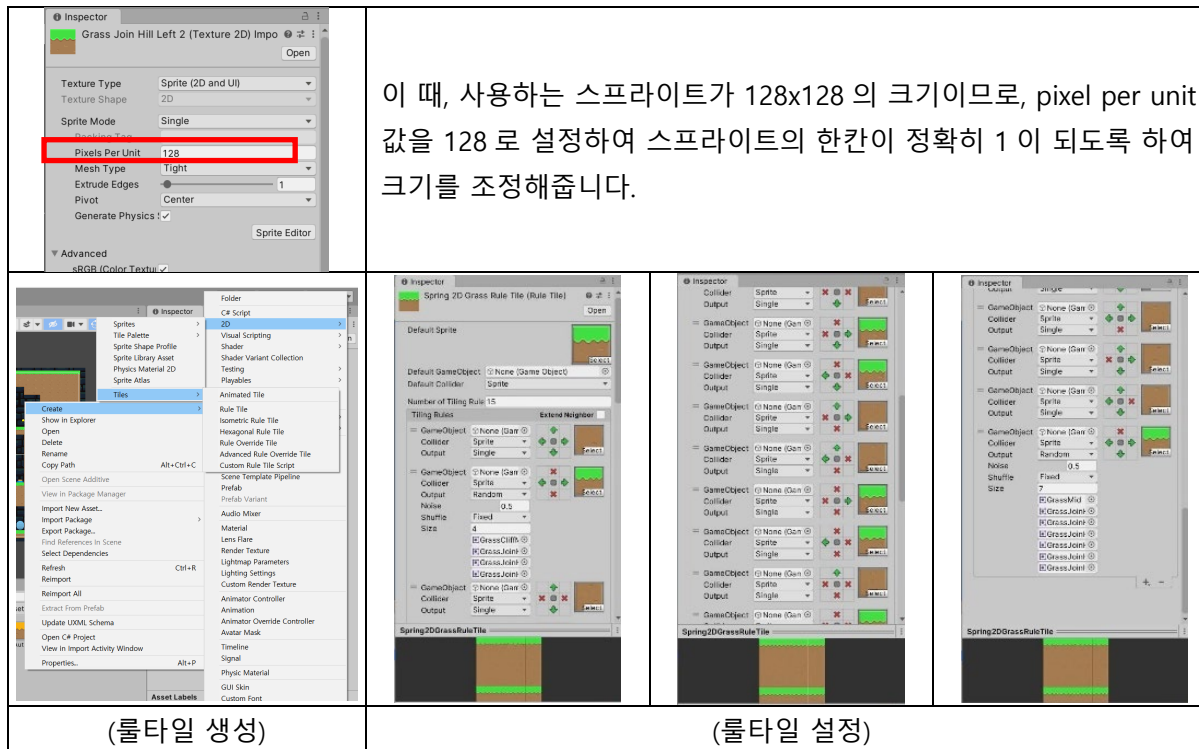
1. 지형 : Tilemap 과 RuleTile 을 이용하여 생성.

※ RuleTile 이란? : 타일 중에서 규칙에 따라 타일의 모양이나 종류를 설정할 수 있는 타일.

※ Tilemap 이란? : 맵을 그릴 때, 오브젝트를 일일이 배치하지 않고, 타일을 이용하여 그림 그리듯이 맵을 생성할 수 있는 컴포넌트.

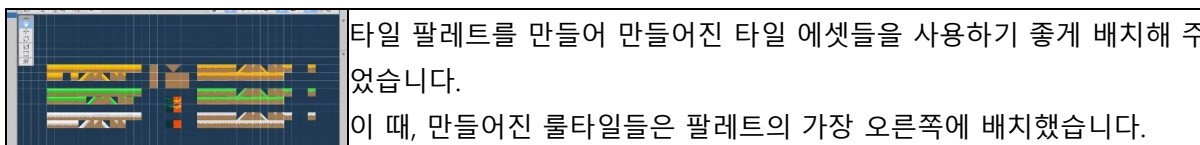


유니티 에셋으로 다운로드 받은 "스프라이트 에셋" 파일을 이용해 "타일 에셋" 파일을 만들어 줍니다.



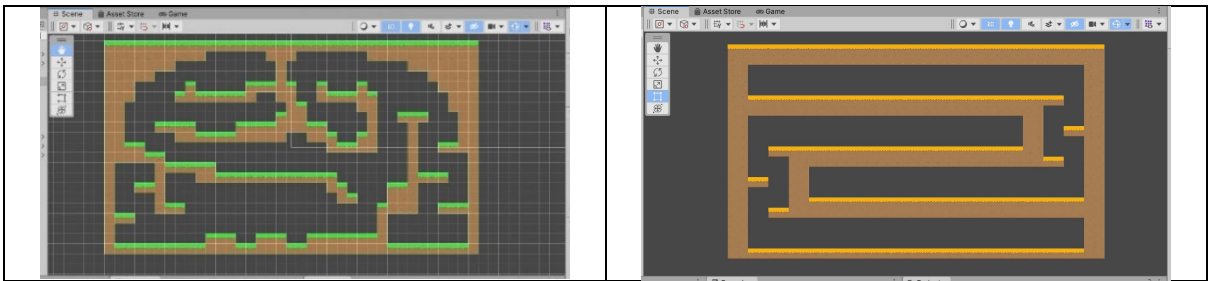
스프라이트들을 이용하여 규칙에 따라 생성될 타일을 정해주고,

이 때, 몇몇 조건일 때는, 여러 스프라이트 중 하나가 랜덤으로 나올 수 있도록 output을 random으로 설정하여 조금 더 다양하고 자연스러운 모양의 맵을 만들 수 있게 하였습니다.



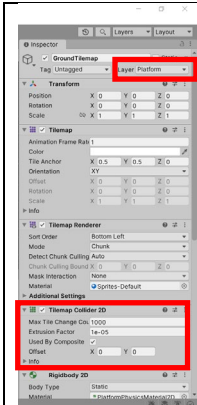
타일 팔레트를 만들어 만들어진 타일 에셋들을 사용하기 좋게 배치해 주었습니다.

이 때, 만들어진 룰타일들은 팔레트의 가장 오른쪽에 배치했습니다.

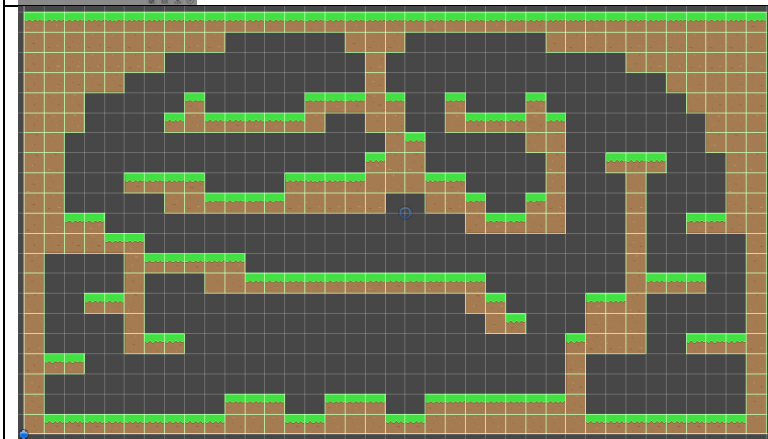


(타일맵과 툴타일을 이용해 생성된 지형들)

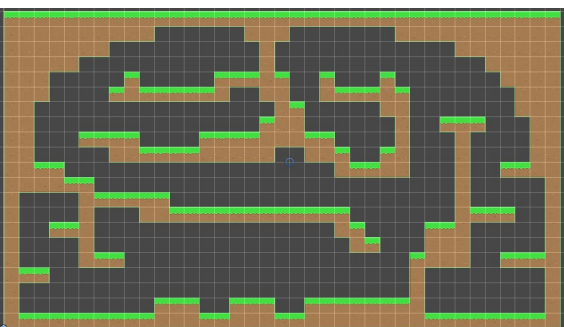
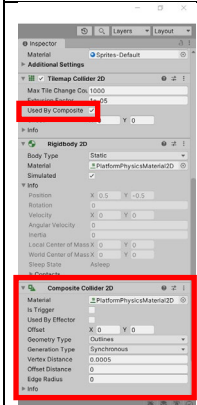
이제, 타일맵을 통해 맵을 그려주었으니, 진짜 지형처럼 플레이어가 밟고 올라설 수 있게 만들어 주겠습니다.



“Platform” 레이어를 만들어, 타일맵 오브젝트에 적용해주고, 캐릭터가 올라설 수 있게, 콜라이더를 적용해줍니다.
이 때, 타일맵이기 때문에, 타일맵 콜라이더2D를 적용해 줍니다.

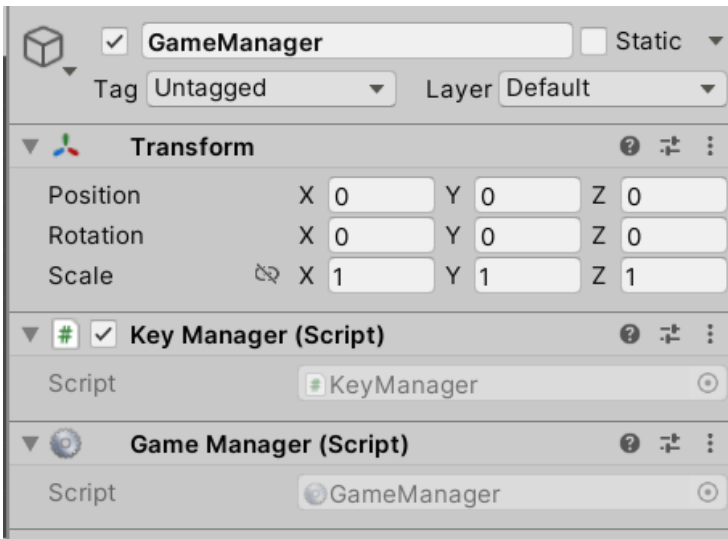


이 때, 단순히 타일맵 콜라이더만 적용해주면, 그려진 각각의 타일마다 콜라이더가 적용됩니다.
하지만, 이는 불필요한 메모리 낭비가 일어날 수도 있고, 가끔 충돌 문제에 의해 끼임 현상이 일어날 수 있습니다.



이를 방지해주기 위해서, Composite Collider 2D를 적용해 주어 각각의 타일에 적용되었던 콜라이더들이 하나의 외곽선에 콜라이더가 적용되도록 하여 해결했습니다.

2. 게임 진행을 위한 GameManager & 키입력을 위한 KeyManager & 효과음 재생을 위한 SfxManager



2-1. 게임 매니저 코드는 싱글톤으로 구현되어 전체 프로젝트에 하나만 존재하도록하여 이를 참조하며 사용하도록 했습니다.

```
public static GameManager instance = null;
```

GameManager를 하나로 참조할 수 있도록 static GameManager instance 를 선언하였고,

```
private void Awake()
{
    if (instance == null)
    {
        instance = this;
        Init(); // 값 초기화.
    }
    else
    {
        // 같은 씬으로 다시 이동하면, Awake()문이 실행되며 오브젝트가 또 생성
        Debug.LogWarning("씬에 두 개 이상의 게임매니저가 존재합니다!");
        Destroy(gameObject); // 중복되므로 하나만 남도록 지워줌.
        return;
    }

    instance = this;
    DontDestroyOnLoad(gameObject); // 이 오브젝트가 파괴되지 않도록 처리.
}
```

Awake문에서 하나만 존재하도록 instance에 자기자신을 참조하도록 처리했고,
2개이상 생성되면 하나만 남도록 나머지를 파괴해주었습니다.

또한, DontDestroyOnLoad(gameObject) 를 이용해 씬을 넘어가더라도
게임 매니저 오브젝트를 파괴하지 않도록 처리해주었습니다.

참조 3개 public bool IsGameOver { get; private set; } // 게임 오버 상태인지 (게임 오버 시 true) 참조 4개 public bool IsClear { get; private set; } // 게임 클리어 상태인지 (클리어 시 true) 참조 8개 public bool IsPause { get; private set; } // 일시정지 상태인지 (일시정지 시 true) 참조 6개 public bool IsPlayingMiniGame { get; private set; } // 미니게임 중인지 (미니게임 중일 때 true) 참조 2개 public int LevelCount { get; private set; } // 스테이지의 개수.	// 게임의 상태에 대한 정보 초기화. 참조 4개 public void Init() { instance.IsGameOver = false; instance.IsClear = false; instance.IsPause = false; instance.IsPlayingMiniGame = false; LevelCount = 2; //임시로 2스테이지까지만, instance.playerAudioSource = instance.gameObject.transform.GetChild(0).GetChild(0).GetComponent<AudioSource>(); // 플레이어 전용 오디오소스 초기화.
// 게임 오버. 참조 1개 public void GameOver() { IsGameOver = true; GamePause(); } // 게임 클리어. 참조 1개 public void GameClear() { IsClear = true; GamePause(); } // 게임 일시정지. 참조 8개 public void GamePause() { IsPause = true; Time.timeScale = 0f; // 시간 흐르는 속도 0으로. (멈춤) playerAudioSource.mute = true; // 플레이어 소리 끄. } // 게임 재개. (일시정지 해제). 참조 5개 public void GameResume() { IsPause = false; Time.timeScale = 1f; // 시간 흐르는 속도 1으로. (1배속) playerAudioSource.mute = false; // 플레이어 소리 켜. }	// 미니게임 시작. 참조 1개 public void StartMiniGame() { IsPlayingMiniGame = true; Time.timeScale = 0f; // 시간 흐르는 속도 0으로. (멈춤) playerAudioSource.mute = true; // 플레이어 소리 끄. } // 미니게임 종료. 참조 1개 public void StopMiniGame() { IsPlayingMiniGame = false; Time.timeScale = 1f; // 시간 흐르는 속도 1으로. (1배속) playerAudioSource.mute = false; // 플레이어 소리 켜. } // 게임 종료. (프로그램 종료) 참조 1개 public void GameQuit() { //Debug.Log("모든 시간 기록 삭제"); //PlayerPrefs에 저장된 모든 데이터 삭제. //PlayerPrefs.DeleteAll(); Debug.Log("게임종료"); // 프로그램 종료. Application.Quit(); // 빌드 안하고 테스트 시에는 작동x.
게임 매니저의 코드를 이렇게 작성하였고, 외부에서 이 클래스의 프로퍼티나 메소드, 변수들을 사용할 때에는 GameManager.instance.IsClear / GameManager.instance.GameClear() 처럼 어디서든 사용할 수 있습니다.	

2-2. 키 매니저

// 키 동작의 enum. 참조 10개 public enum KeyAction { LEFT = 0, // 왼쪽. RIGHT, // 오른쪽. JUMP, // 점프. SWAP, // 플레이어 스왑. INTERACTION, // 상호작용. KEYCOUNT // 키 개수. }	// 키 세팅 static 클래스. 키보드 세팅 값을 각 키 동작에 맞게 저장하는 클래스. 참조 8개 public static class KeySetting { // 원하는 동작과 입력값을 딕셔너리로. public static Dictionary<KeyAction, KeyCode>keys = new Dictionary<KeyAction, KeyCode>(); }
---	---

원하는 동작을 알아보기 쉽게 KeyAction이라는 enum을 생성하여 이름을 부여해주었고,

KeySetting이라는 static 클래스를 생성하고, static dictionary<KeyAction,KeyCode>()의 keys 딕셔너리를 생성해 원하는 동작에 원하는 키 값을 연결해주었습니다.

이 KeySetting클래스의 keys 딕셔너리를 이용해 동작에 맞는 키 입력을 설정해줄 수 있고, 사용할 수 있습니다.

```

// Unity 스크립트 (서맨 1개) 1. 스크립트
public class KeyManager : MonoBehaviour
{
    // 디폴트 키 값.
    KeyCode[] defaultKeys = new KeyCode[] {
        KeyCode.LeftArrow, // 왼쪽. (왼쪽화살표(+))
        KeyCode.RightArrow, // 오른쪽. (오른쪽화살표(+))
        KeyCode.Space, // 점프. (스페이스바)
        KeyCode.C, // 플레이어 스왑. (C)
        KeyCode.F // 상호작용. (F)
    };
}

// Unity 스크립트 (서맨 1개)
private void Awake()
{
    KeySetting.keys.Clear(); // 키세팅 초기화.
    // 초기에 키세팅을 디폴트 키값으로 초기화.
    for (int i = 0; i < (int)KeyAction.KEYCOUNT; i++)
    {
        KeySetting.keys.Add((KeyAction)i, defaultKeys[i]);
    }
}

```

KeyManager 클래스에서는 디폴트 키 값으로 키세팅을 초기화해줍니다.

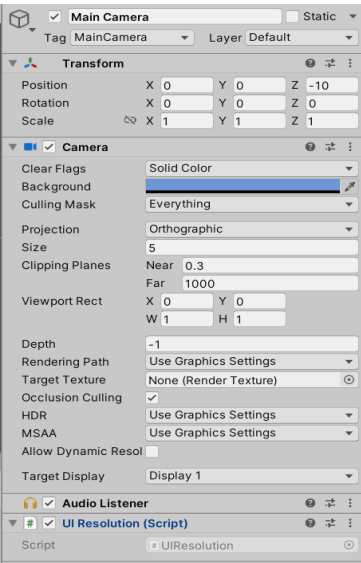
키매니저의 키는, `Input.GetKey(KeySetting.keys[KeyAction.RIGHT])` 처럼 사용할 수 있습니다.

2-3. 효과음 매니저(SfxManager)

 <pre> [Header("[플레이어 관련"]) [SerializeField] private AudioClip walkAudioClip; [SerializeField] private AudioClip jumpAudioClip; [SerializeField] private AudioClip deathAudioClip; [SerializeField] private AudioClip swapAudioClip; [Header("[오브젝트 관련"]) [SerializeField] private AudioClip buttonAudioClip; [SerializeField] private AudioClip switchAudioClip; [SerializeField] private AudioClip doorOpenAudioClip; [SerializeField] private AudioClip doorCloseAudioClip; [Header("[장면 관련"]) [SerializeField] private AudioClip stageStartAudioClip; [SerializeField] private AudioClip stageClearAudioClip; [Header("[퍼즐 관련"]) [SerializeField] private AudioClip puzzleClearAudioClip; [SerializeField] private AudioClip puzzlePieceDragAudioClip; [SerializeField] private AudioClip puzzlePieceDropAudioClip; [SerializeField] private AudioClip puzzlePieceSnapAudioClip; [Header("[UI 관련"]) [SerializeField] private AudioClip uiButtonClickAudioClip; [SerializeField] private AudioClip uiWindowOnAudioClip; [SerializeField] private AudioClip uiWindowOffAudioClip; </pre>	효과음 재생을 위한 SfxManager 입니다. SfxManager.cs 스크립트를 작성하고, 인스펙터에서 각 클립을 변수로 넣어 해당 오디오가 필요할 때 재생됩니다. 해당 인스펙터가 이렇게 나타나는 이유는 [Header] 키워드를 통해 라벨링을 해주어 조금 더 보기 편하게 해주었습니다.
<pre> // 19줄 public Dictionary<ESfxName, AudioClip> SfxDict { get; private set; }; private ESfxName[] sfxNameList; private AudioClip[] sfxClipList; </pre>	SfxManager 역시 KeyManager처럼 딕셔너리를 만들어 관리합니다.
<pre> // AudioClipList와 AudioDict 초기화 메소드. // 1개 public void Init() { InitAudioClipList(); // AudioClipList를 위해 선언한 EAudioSourceName의 값을 가져와서 저장. sfxNameList = (ESfxName[])System.Enum.GetValues(typeof(ESfxName)); // for문을 돌면서, EAudioSourceName의 이름과 AudioClipList의 오디오 소스를 for (int i = 0; i < listLength; i++) { SfxDict[sfxNameList[i]] = sfxClipList[i]; // audioDict 초기화. } playerWalkAudioSource.clip = SfxDict[ESfxName.Walk]; } // 1개 public void InitAudioClipList() { sfxClipList[(int)ESfxName.Walk] = walkAudioClip; sfxClipList[(int)ESfxName.Jump] = jumpAudioClip; sfxClipList[(int)ESfxName.Death] = deathAudioClip; sfxClipList[(int)ESfxName.Swap] = swapAudioClip; sfxClipList[(int)ESfxName.Button] = buttonAudioClip; sfxClipList[(int)ESfxName.Switch] = switchAudioClip; sfxClipList[(int)ESfxName.DoorOpen] = doorOpenAudioClip; sfxClipList[(int)ESfxName.DoorClose] = doorCloseAudioClip; sfxClipList[(int)ESfxName.StageStart] = stageStartAudioClip; sfxClipList[(int)ESfxName.StageClear] = stageClearAudioClip; sfxClipList[(int)ESfxName.PuzzleClear] = puzzleClearAudioClip; sfxClipList[(int)ESfxName.PuzzlePieceDrag] = puzzlePieceDragAudioClip; sfxClipList[(int)ESfxName.PuzzlePieceDrop] = puzzlePieceDropAudioClip; sfxClipList[(int)ESfxName.PuzzlePieceSnap] = puzzlePieceSnapAudioClip; sfxClipList[(int)ESfxName.UIButtonClick] = uiButtonClickAudioClip; sfxClipList[(int)ESfxName.UIWindowOn] = uiWindowOnAudioClip; sfxClipList[(int)ESfxName.UIWindowOff] = uiWindowOffAudioClip; } </pre>	딕셔너리값과 배열을 초기화하는 코드입니다. 각 역할에 맞게 오디오클립을 넣어줍니다.

<pre> 참조 1개 public void PlayWalkSound() { if (!playerWalkAudioSource.isPlaying) { playerWalkAudioSource.Play(); } } // 걷기 사운드 종료 메소드. 참조 3개 public void StopWalkSound() { if (playerWalkAudioSource.isPlaying) { playerWalkAudioSource.Stop(); } } </pre>	플레이어 걷기는 단발성 효과음이 아니라 걷는 중에 반복적으로 재생되어야 하기 때문에, Play() 와 Stop() 을 이용하여 효과음을 재생하고 정지합니다.
<pre> // AudioManager의 오디오 소스에 해당 사운드 실행하는 메소드. // 점프 사운드 실행 메소드. 참조 1개 public void PlayJumpSound() { sfxAudioSource.PlayOneShot(SfxDict[ESfxName.Jump]); } </pre>	나머지 효과음은 단발성 효과음이기 때문에, PlayOneShot()을 이용하여 효과음을 재생합니다.
외부에서 효과음을 재생하기 위해서는 sfxManager를 선언하고 GameManager의 오브젝트의 자식으로 할당하고, sfxManager.PlayJumpSound() 처럼 사용하면 외부 스크립트에서도 효과음을 쉽게 재생할 수 있습니다.	

3. 해상도 고정 (1920x1080으로 고정) (UIResolution.cs)



```

private void Update()
{
    int deviceWidth = Screen.width; // 기기 너비 저장.
    int deviceHeight = Screen.height; // 기기 높이 저장.

    // 현재 기기의 너비나 높이가 변경되면,
    if (currentDeviceWidth != deviceWidth || currentDeviceHeight != deviceHeight)
    {
        // 변경된 너비/높이로 재설정하고,
        currentDeviceWidth = deviceWidth;
        currentDeviceHeight = deviceHeight;

        SetResolution(); // 창 크기 변경시에도 게임 해상도 고정.
    }
}

```

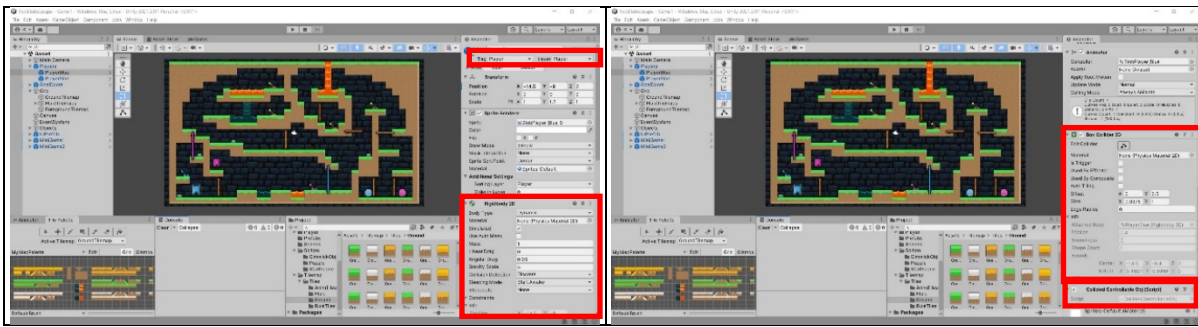
'[Unity] 해상도에 따른 화면 비율 고정하기' (<https://giseung.tistory.com/19>) 를 참고하여, UIResolutuin.cs 를 작성하고, Main Camera에 추가해주었습니다. Main Camera의 설정은 이미지와 같이 했습니다.

해상도가 변경될 시에도, 원활한 동작을 위해 Update()문 안에도 SetResolution문을 실행시키고,

이 때, 해상도가 변경되었을 때를 탐지하기 위해서, if문을 작성하고, 변경 시 변경된 해상도를 저장하고 해상도를 맞춰줍니다.

4. 플레이어블 캐릭터

우선 2개의 플레이어블 캐릭터를 만들어야 하기 때문에, 각각의 캐릭터에 대한 설정을 해줍니다.

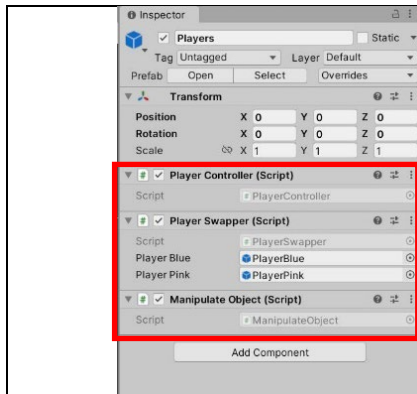


플레이어를 위해 "Player"라는 레이어와 태그를 각각 만들어주고, 각 캐릭터에게 적용시켰습니다.

플레이어의 충돌(바닥을 밟고 서있을 수 있게 + 다른 물체와의 상호작용을 위해 필요)을 위해서 Rigidbody2D 와 Collider2D를 추가해주고, 또한, 나중에 나올 오브젝트들과의 상호작용을 위해, 충돌한 상호작용 가능한 오브젝트를 저장(관리)하는 스크립트를 작성하여, 추가해주었습니다.

<pre>// 자신과 충돌한 조작가능한 오브젝트를 관리(저장)하는 클래스. @ Unity 스크립트 (자신 참조 2개) 참조 1개 public class CollidedControllableObj : MonoBehaviour</pre>	CollidedControllableObj 클래스
<pre>private LinkedList<GameObject> collidedObjects; // 충돌한 오브젝트들 리스트. 참조 1개 public LinkedList<GameObject> GetCollidedObjects() { return collidedObjects; }</pre>	충돌한 (상호작용 가능한)오브젝트들을 담을 수 있는 리스트 선언.
<pre>// collider(trigger) 가 접촉되면, @ Unity 메시지 참조 0개 private void OnTriggerEnter2D(Collider2D collision) { // Debug.Log("OnTriggerEnter2D : " + collision.name); // 충돌한 오브젝트에 IControlObject를 상속받는 컴포넌트가 있는지 확인해서 있으면, if (collision.TryGetComponent<IControlObject>(out _)) { // '충돌된 오브젝트들 리스트'에 충돌한 해당 오브젝트를 추가. collidedObjects.AddLast(collision.gameObject); } }</pre>	OnTriggerEnter2D()를 이용하여 충돌했을 때, 이벤트를 발생시키고, 충돌한 오브젝트에 대해서, TryGetComponent<IControlObject>()를 이용해서, IControlObject 스크립트를 상속받는 컴포넌트가 있는지 확인해서, 있으면, 리스트에 추가합니다.
<pre>// collider(trigger) 가 접촉이 해제되면, @ Unity 메시지 참조 0개 private void OnTriggerExit2D(Collider2D collision) { // Debug.Log("OnTriggerExit2D : " + collision.name); // 충돌 해제된 오브젝트가 리스트에 있으면, 리스트에서 지움. if (collidedObjects.Contains(collision.gameObject)) { collidedObjects.Remove(collision.gameObject); } }</pre>	OnTriggerExit2D()를 이용하여 충돌 해제했을 때, 이벤트를 발생시키고, 충돌 해제된 오브젝트에 대해서, Contains()를 이용해서, 리스트에 있는지 확인해서, 있으면, 리스트에서 제거합니다.

다음, 각각의 캐릭터 오브젝트 상위에 Players라는 오브젝트를 생성하고,
플레이어들을 조작하기 위한 설정을 해줍니다.



두 플레이어 오브젝트의 상위에 Players라는 빈 오브젝트를 만들어,
플레이어 조작(이동, 점프, 죽음),
플레이어 변경,
오브젝트 조작
을 위한 스크립트들을 각각 작성하여 넣어주었습니다.
다음은 플레이어를 위한 코드들입니다.

```
// 플레이어 이동.
참조 1개
private void Move()
{
    Vector2 direction = Vector2.zero;
    // 키보드 입력 받아서 방향값을 설정. ①
    if (Input.GetKey(KeyCode.RightArrow))
    {
        direction = Vector2.right;
    }
    else if (Input.GetKey(KeyCode.LeftArrow))
    {
        direction = Vector2.left;
    }

    GameObject nowPlayer = playerSwapper.GetNowPlayer();
    Rigidbody2D nowPlayerRigid = nowPlayer.GetComponent<Rigidbody2D>(); ②
    // 현재 플레이어의 속도를 설정한 방향값에 이동속도로 설정하여, 현재 플레이어를 이동시키는 코드. ③
    nowPlayerRigid.velocity = new Vector2(direction.x * speed, nowPlayerRigid.velocity.y);

    // 걷기 애니메이션.
    // 방향값이 있으면 이동 중이므로, 걷기 애니메이션 실행.
    if (direction != Vector2.zero)
    {
        nowPlayer.GetComponent<SpriteRenderer>().flipX = (direction == Vector2.left); // 스프라이트 방향 설정. 플레이어의 방향이 왼쪽이면 스프라이트를 flip함.
        nowPlayer.GetComponent<Animator>().SetBool("isWalking", true);

        if (IsLanding()) // 땅에 있을 때,
        {
            // 걷기 효과를 재생.
            sfxManager.PlayWalkSound();
        }
    }
    else
    {
        // 그렇지 않으면, 정지이므로, 걷기 애니메이션 끄기.
        nowPlayer.GetComponent<Animator>().SetBool("isWalking", false);

        // 걷기 효과를 중지.
        sfxManager.StopWalkSound();
    }
}
```

<플레이어 이동>

- ①: 키보드에서 입력 값을 받아서 플레이어의 방향 값을 설정해 줍니다.
- ②: 현재 조작하고 있는 플레이어를 가지고 와서 Rigidbody를 가져옵니다.
- ③: 설정한 방향 값에 맞게 현재 플레이어의 속도를 설정하여 플레이어를 이동시키는 코드입니다.
- ④: 마지막으로 플레이어 이동 시 애니메이션 및 걷기 효과음을 재생시키는 부분입니다.

```
// 플레이어 점프.
참조 1개
private void Jump()
{
    // 바닥을 밟으면, ①
    if (IsLanding())
    {
        StartCoroutine(JumpCoroutine(playerSwapper.GetNowPlayer()));
    }
}

// 점프 코루틴.
참조 1개
private IEnumerator JumpCoroutine(GameObject nowPlayer)
{
    // 점프 효과음 재생.
    sfxManager.PlayJumpSound(); ②

    // 점프 애니메이션 실행.
    nowPlayer.GetComponent<Animator>().SetBool("isJumping", true);

    Rigidbody2D nowPlayerRigid = nowPlayer.GetComponent<Rigidbody2D>();
    // 플레이어가 점프하기 직전에 y축의 속도를 0으로 설정.
    nowPlayerRigid.velocity = new Vector2(nowPlayerRigid.velocity.x, 0f);
    // 플레이어를 위의 방향으로 힘을 주어 점프 시킴.
    nowPlayerRigid.AddForce(Vector2.up * jumpPower, ForceMode2D.Impulse);

    yield break;
}
```

<플레이어 점프>

- ①: 바닥을 밟고 있는지 판단하고, 바닥을 밟고 있는 상태이면, 점프 코루틴을 재생합니다.
현재 조작하고 있는 플레이어만 점프하도록 매개변수로 현재 플레이어를 넘겨줍니다.
- ②: 점프 코루틴이 실행되면, 효과음과 애니메이션을 재생하고, 점프력이 일정하게 적용되기 위해서 속도를 0으로 설정한 다음, AddForce()로 위의 방향으로 힘을 주어 점프를 시킵니다.

```
// 플레이어 죽음.
참조 2개
public void Die()
{
    Debug.Log("플레이어 죽음");

    // 걷기 효과음 종료.
    sfxManager.StopWalkSound();
    // 죽음 효과음 재생.
    sfxManager.PlayDeathSound();

    PlayerDeathAnim();

    GameManager.instance.GameOver(); // 게임매니저에 게임오버 알림.
}
```

<플레이어 죽음>

플레이어가 죽어도 만약 걷고 있었다면, 걷는 소리가 계속 재생되므로, 걷기 효과음을 종료하고, 죽음 효과음을 재생합니다. 그 다음 플레이어 죽음 애니메이션을 실행하고, 게임매니저에 게임오버를 알립니다.

```
// 플레이어 죽음 시 플레이어를 부드럽게 사라지도록 하는 코루틴.
참조 1개
private IEnumerator PlayerDeathAnimCoroutine()
{
    Color fromColor = new Color(1f, 1f, 1f, 1f); // 흰색 불투명.
    Color toColor = new Color(0f, 0f, 0f, 0f); // 검은색 투명.

    GameObject playerBlue = playerSwapper.GetPlayerBlue();
    GameObject playerPink = playerSwapper.GetPlayerPink();

    SpriteRenderer blueSpriteRenderer = playerBlue.GetComponent<SpriteRenderer>();
    SpriteRenderer pinkSpriteRenderer = playerPink.GetComponent<SpriteRenderer>();

    float time = 0f;
    float duration = 1f;
    while (time < duration)
    {
        // Lerp 함수를 이용하여 부드러운 색 변경이 가능하게 함.
        blueSpriteRenderer.color = Color.Lerp(fromColor, toColor, time / duration);
        pinkSpriteRenderer.color = Color.Lerp(fromColor, toColor, time / duration);

        time += Time.unscaledDeltaTime;

        yield return null;
    }

    // while 문 종료시 확실하게 toColor로 변경되도록 함.
    blueSpriteRenderer.color = toColor;
    pinkSpriteRenderer.color = toColor;

    // 스프라이트 이미지 상으로 사라지게 한 다음 오브젝트를 비활성화.
    playerBlue.SetActive(false);
    playerPink.SetActive(false);
}
```

<플레이어 죽음 애니메이션 코루틴>

플레이어가 죽는 경우, 자연스럽게 어두워지면서 투명해지도록 하고, 캐릭터 이미지를 사라지게 하는 코드입니다. 캐릭터 스프라이트 렌더러를 가져와 캐릭터의 색변경을 하였고, 부드러운 색 변경을 위해 Lerp() 함수를 이용했습니다. 색 변경이 끝나면, SetActive(false)를 통해 오브젝트를 비활성화해주었습니다.

```
// 플레이어 착지.
참조 1개
private void LandingPlatform()
{
    GameObject nowPlayer = playerSwapper.GetNowPlayer();
    Animator nowPlayerAnimator = nowPlayer.GetComponent<Animator>();

    if (IsLanding())
    {
        // 바닥을 밟으면, 애니메이션의 "isJumping"값을 false로 설정.
        nowPlayerAnimator.SetBool("isJumping", false);
    }
    else
    {
        // 바닥을 밟지 않으면, -> 공중이면, 애니메이션의 "isJumping"값을 true로 설정. -> 점프가 아닌 추락할 때도 점프로 처리하기
        nowPlayerAnimator.SetBool("isJumping", true);
    }
}
```

<플레이어 착지 처리>

플레이어가 바닥을 밟으면 착지 상태로 처리를 하고, 아니면 공중으로 판단해서, 점프 상태로 처리하는 코드입니다. 이는 점프키를 누르지 않고 추락하는 경우에도 애니메이션을 재생시킬 수 있도록 하기 위해서, 공중이면 점프로 처리하도록 작성했습니다.

```
참조 3개
private bool IsLanding()
{
    GameObject nowPlayer = playerSwapper.GetNowPlayer();

    // 바닥 감지 영역을 위한 값들 초기화.
    float detectionWidth = 0.5f * nowPlayer.GetComponent<Collider2D>().bounds.size.x; // 감지 영역 너비를 플레이어의 클라이
    Vector2 hitPos = nowPlayer.transform.position; // 감지 영역 위치.
    Vector2 hitSize = new Vector2(detectionWidth, 0.1f); // 감지 영역 크기.

    // BoxCast로 박스모양의 영역으로 raycastHit. (레이어가 Platform인 오브젝트만 감지)
    RaycastHit2D raycastHit2D = Physics2D.BoxCast(hitPos, hitSize, 0f, Vector2.down, 0, LayerMask.GetMask("Platform"));

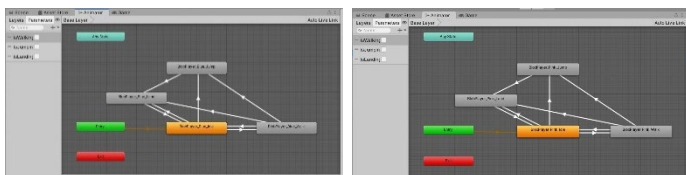
    // 바닥을 밟으면, collider가 null이 아님.
    if (raycastHit2D.collider != null)
    {
        return true;
    }
    return false;
}
```

<플레이어 착지 검사(바닥 감지)>

RaycastHit2D의 BoxCast() 를 이용하여 "Platform" 레이어만을 감지하도록 했습니다. 만약 해당 레이어가 감지되면, 바닥에 닿은 것으로 판단해서 true를 반환합니다.

<pre>// (Debug)를 위한 해당 오브젝트 선택 시 gizmo를 그려주는 메소드. @OnlyOnBuild private void OnDrawGizmosSelected() { GameObject nowPlayer = playerSwapper.GetNowPlayer(); // 바닥 감지 영역을 위한 값을 초기화. float detectionWidth = 0.5f * nowPlayer.GetComponent<Collider2D>().bounds.size.x; // 감지 영역 너비를 플레이어의 콜라이더 Vector2 hitPos = nowPlayer.transform.position; // 감지 영역 위치. Vector2 hitSize = new Vector2(detectionWidth, 0.1f); // 감지 영역 크기. // BoxCast로 박스모양의 영역으로 raycastHit. (레이어가 Platform인 오브젝트만 감지) RaycastHit2D raycastHit2D = Physics2D.BoxCast(hitPos, hitSize, 0f, Vector2.down, 0, LayerMask.GetMask("Platform")); // 충돌 감지 시각화를 위한 DrawRay. 바닥을 밟으면 녹색, 아니면 빨간색으로 씬에서 볼 수 있음. Gizmos.color = (raycastHit2D.collider != null ? Color.green : Color.red); Gizmos.DrawCube(hitPos, hitSize); }</pre> <착지 감지 영역 디버그>	 <기즈모 이미지> (※ Start씬에서 실행하고 값이 초기화되고나서 Game1(Game2) 씬으로 넘어가야, 실행 중에 Players 오브젝트를 선택 하면 기즈모를 볼 수 있 습니다.)	OnDrawGizmosSelected() 메소드는 해당 오브젝트를 선택했을 때, Scene 창에서 기즈모를 볼 수 있습니다. BoxCast를 사용했었으므로, Gizmos.DrawCube()를 사용하여 박스를 그려주었습니다. 이 때, 시각적으로 바닥에 닿았는지 아닌지 판단하는 것을 돕기 위해서 color를 지정해주었습니다.
<pre>// 플레이어 스왑(파랑이 <-> 분홍이). 참조 1개 void SwapPlayer() { // 이전 플레이어의 속도를 0으로 설정. nowPlayer.GetComponent<Rigidbody2D>().velocity = Vector2.zero; // 이전 플레이어의 애니메이션들 종료. nowPlayer.GetComponent<Animator>().SetBool("isWalking", false); // 걷기. nowPlayer.GetComponent<Animator>().SetBool("isJumping", false); // 점프. // 이전 플레이어의 색을 비활성화된 색으로 변경. nowPlayer.GetComponent<SpriteRenderer>().color = deactivatedColor; // 이전 플레이어의 Order in Layer를 0으로 변경(이전 플레이어가 현재 플레이어의 뒤로 가도록). nowPlayer.GetComponent<SpriteRenderer>().sortingOrder = 0; // 플레이어 변경. nowPlayer = (nowPlayer == playerBlue) ? playerPink : playerBlue; // 현재 플레이어의 색을 활성화된 색으로 변경. nowPlayer.GetComponent<SpriteRenderer>().color = activatedColor; // 현재 플레이어의 Order in Layer를 1으로 변경(현재 플레이어가 이전 플레이어의 앞으로 가도록). nowPlayer.GetComponent<SpriteRenderer>().sortingOrder = 1; }</pre> <플레이어 스왑>		현재 플레이어를 파랑이와 분홍이를 바꾸주는 코드입니다. 이 때, 시각적으로 보여주기 위해서, 색도 바꿔주었고, 플레이어 변경 시 이전 플레이어의 애니메이션이 계속 재생되는 것을 방지하기 위해서, 이전 플레이어의 애니메이션은 종료했습니다. 현재 조종하는 캐릭터가 이전 플레이어의 위로 오도록 layer의 순서도 설정해주었습니다.
<pre>// 조작 가능한 오브젝트 조작 코루틴. 참조 1개 private IEnumerator ManipulateObjCoroutine() { isControlling = true; // 현재 플레이어의 CollidedControllableObj 스크립트의 충돌중인 조작가능한 오브젝트들 리스트를 가져옴. LinkedList<GameObject> objCollider = playerSwapper.GetNowPlayer().GetComponent<CollidedControllableObj>().GetCollidedObjects(); // 만약 현재 플레이어가 조작가능한 오브젝트와 하나 이상 충돌중이라면, null이 아니거나 카운트가 0보다 큼. if (objCollider != null objCollider.Count > 0) { // 충돌중인 조작가능한 오브젝트 하나하나 반복하며. foreach (GameObject obj in objCollider) { // 해당 오브젝트의 IControlObject를 상속하는 컴포넌트를 모두 가져옴. (하나의 오브젝트에 여러 기능이 있을 수 있으므로 GetComponent()를 여러번 호출) foreach (IControlObject controllableObj in obj.GetComponent<IControlObject>()) { // 가져온 모든 컴포넌트들 각각의 ControlObject()를 실행. controllableObj.ControlObject(); // 해당 오브젝트의 각 기능을 조작하는 메소드 실행. } } isControlling = false; yield break; } }</pre> <플레이어 오브젝트 조작>		각 캐릭터에 적용했던, ColliderControllableObj 컴포넌트의 리스트들을 가져와 해당 리스트의 오브젝트들에 대한 오브젝트 조작을 실행하는 코드입니다. 이 때, 여러 오브젝트가 겹치거나, 한 오브젝트가 여러 동작을 할 수 있으므로, for문과 GetComponent() 함수를 이용하였습니다. ControlObject() 메소드는 IControlObject 인터페이스의 메소드로 각기 다른 오브젝트마다의 동작을 하나로 통일해서 실행 시키기 위해서 만들었습니다.

다음은 플레이어의 애니메이션 실행을 위한 부분입니다.



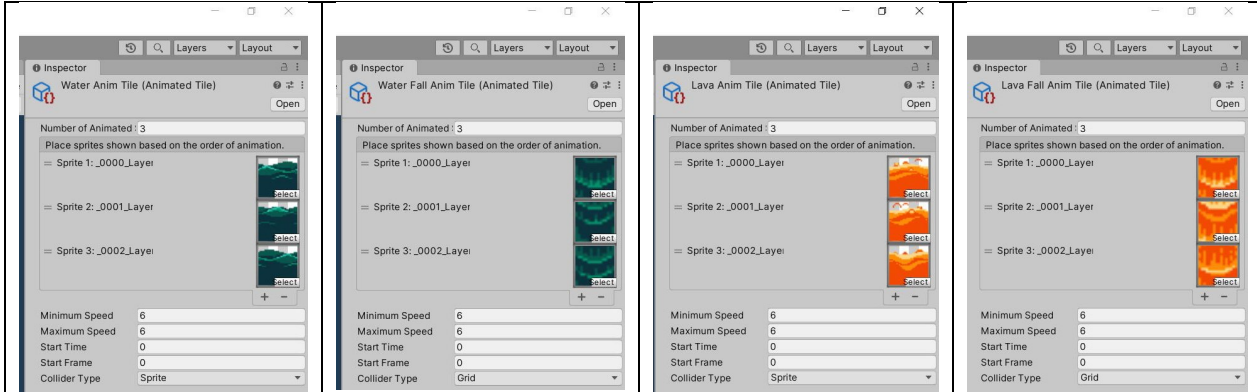
플레이어의 이동과 점프, 착지 시마다 애니메이션을 적용해주기 위해 애니메이션 클립을 이용하여 애니메이션 컨트롤러로 FSM(유한상태머신)을 만들어주고, 이는 한 상태에 한 애니메이션만 적용될 수 있도록 도와줍니다. 애니메이션 클립은 유니티 에셋을 이용하였고, 컨트롤러를 작성하여 상태와 조건에 따라 올바른 애니메이션이 재생되도록 해주었습니다.

5. 물 & 용암

Animated Tile 을 이용하여, 물과 용암이 흐르는 애니메이션이 적용되어 움직이는 타일을 만들어주었습니다.

만들어진 물과 용암 Animated Tile 에셋도 아까 만들어 놓은 타일 팔레트에 추가하여 타일맵으로 그릴 수 있도록 해주었습니다.

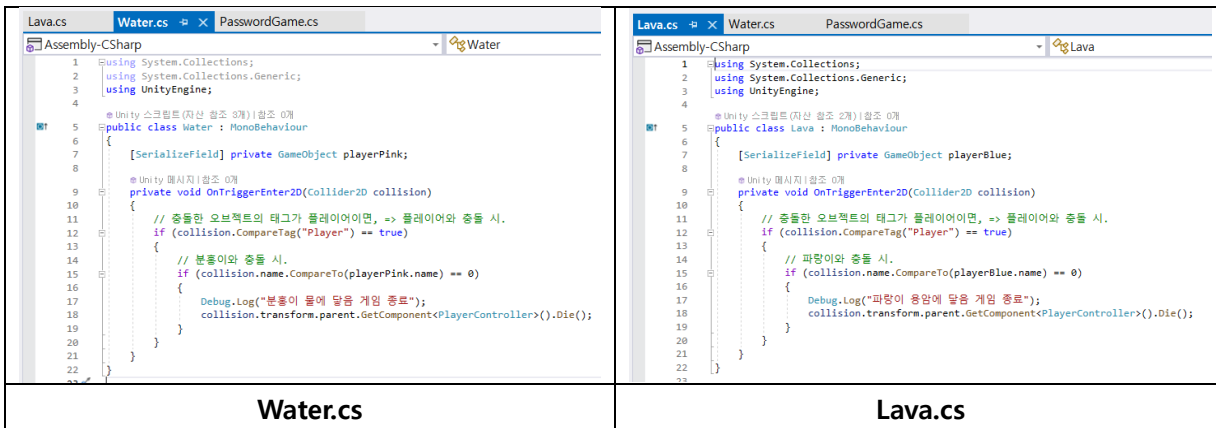
또한 애니메이션의 적절한 재생 속도를 위해 speed 값을 조정해주었습니다.



Animated Tile의 설정을 위한 인스펙터 창입니다. 원하는 스프라이트를 집어넣으면, 1->2->3->1 순서로 반복하며, 재생되는 타일이 만들어질 수 있습니다.

물과 용암은 각각의 속성과 성질을 가져야 하기 때문에, 각각의 타일맵으로 만들어 주었습니다.

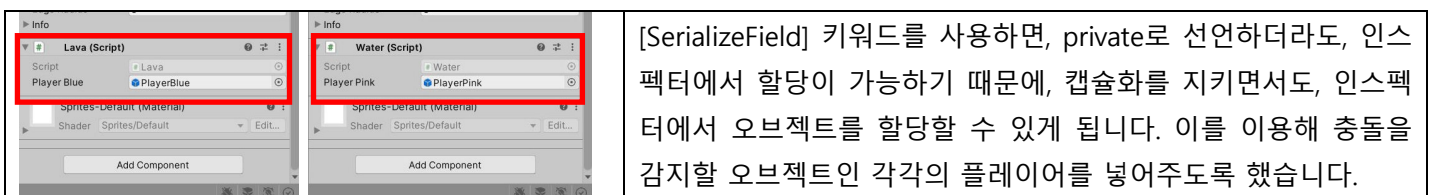
그리고, 물에는 Water.cs 스크립트, 용암에는 Lava.cs 스크립트를 작성하여 적용해주어,



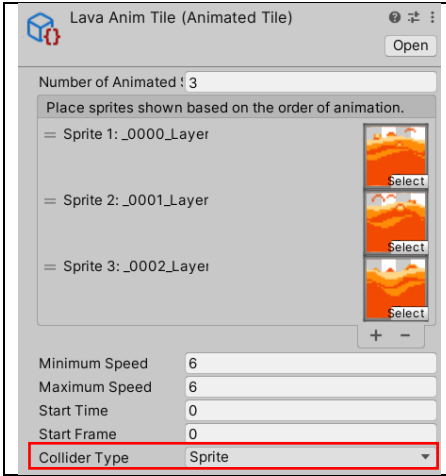
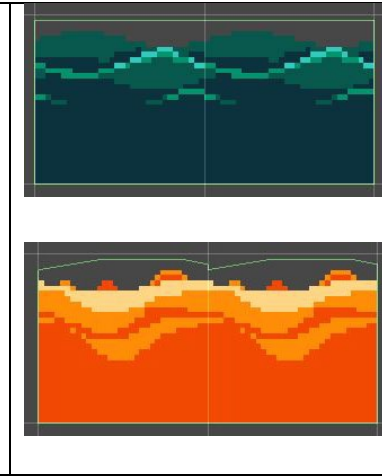
물에 분홍기가 닿거나, 용암에 파랑기가 닿으면, 플레이어가 죽을 수 있도록 코드를 작성했습니다.

OntriggerEnter2D() 메소드를 활용하여 충돌 이벤트 발생 시,

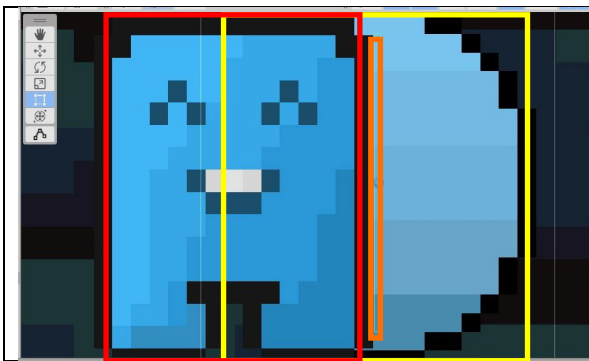
태그를 이용하여 플레이어인지 감지하고, 플레이어라고 판단되면, 인스펙터에서 넣은 플레이어의 이름과 충돌한 오브젝트와의 이름을 비교하여 같으면, 닿았다고 판정하고 PlayerController컴포넌트의 Die()메소드를 통해 플레이어를 죽게 합니다.



[SerializeField] 키워드를 사용하면, private로 선언하더라도, 인스펙터에서 할당이 가능하기 때문에, 캡슐화를 지키면서도, 인스펙터에서 오브젝트를 할당할 수 있게 됩니다. 이를 이용해 충돌을 감지할 오브젝트인 각각의 플레이어를 넣어주도록 했습니다.

		또한, 용암과 물 타일의 콜라이더 타입을 Sprite로 지정하여, 캐릭터가 물이나 용암 옆에 있더라도 빠지지만 않으면 죽지 않도록 처리를 했습니다.
---	--	---

6. 목표 지점 (Goal)

	빨강 - 캐릭터 스프라이트 & 콜라이더 범위 노랑 - 목표지점 스프라이트 범위 주황 - 목표지점 콜라이더 범위 콜라이더의 범위를 다음과 같이 지정하여, 플레이어가 목표지점의 중간 정도까지 걸쳐야 충돌에 감지되도록 하여, 스프라이트끼리 맞닿았을 때, 바로 게임 클리어 처리가 되는 것을 방지했습니다.
<pre> 1 void OnTriggerEnter2D(Collider2D collision) 2 { 3 if (collision.gameObject.CompareTag("Player")) 4 { 5 if (collision.gameObject.name.CompareTo(Player.name) == 0) 6 { 7 isGoal = true; 8 } 9 } 10 } </pre>	<pre> 1 void OnTriggerExit2D(Collider2D collision) 2 { 3 if (collision.gameObject.CompareTag("Player")) 4 { 5 if (collision.gameObject.name.CompareTo(Player.name) == 0) 6 { 7 isGoal = false; 8 } 9 } 10 } </pre>
플레이어와의 충돌을 감지하기 위해서. OnTriggerEnter2D() / OnTriggerExit2D() 를 이용하여, 태그와 충돌체의 이름이 플레이어와 같은지 비교하여 처리했습니다.	

7. 오브젝트 조작 관련.

7-1. 조작 오브젝트 인터페이스

```

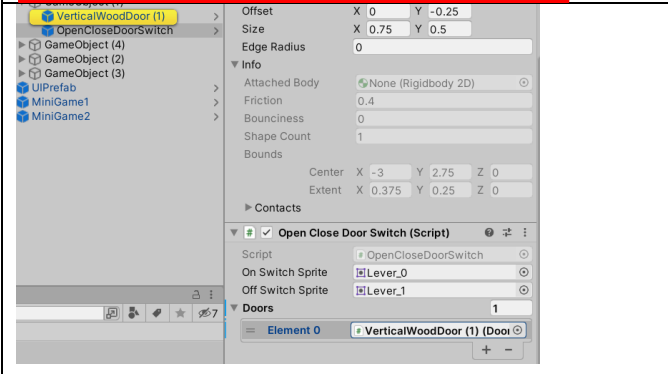
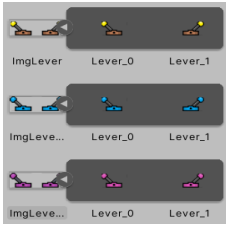
1  // 조작하는 오브젝트에 들어갈 조작 인터페이스.
   참조 4개
2  public interface IControlObject
3  {
4      // 참조 2개
       public void ControlObject();    // 조작 인터페이스의 오브젝트 조작 메소드.
5  }
6
        
```

스위치에 들어가는 인터페이스로 추후에 조작 오브젝트가 늘어난다면, 해당 인터페이스를 상속받게 하면 됩니다.

현재는 스위치 오브젝트 하나만 이 인터페이스를 상속받고 있습니다.

상호작용키(F) 키를 누르면 이 인터페이스의 ControlObject()가 실행되도록 작성했습니다.

7-2.스위치 오브젝트

<pre>// (조작가능한 오브젝트인 스위치)와 상위 추상 클래스. @Unity 스크립트 참조: 2개 public abstract class SwitchObject : MonoBehaviour, IControlObject { private SfxManager sfxManager; @Unity 메시지 참조: 0개 private void Start() { sfxManager = GameManager.Instance.gameObject.transform.GetChild(0).GetComponent<SfxManager>(); } 참조: 2개 public void ControlObject() { sfxManager.PlaySwitchSound(); ControlSwitch(); // 스위치를 조작하는 메소드 실행. } 참조: 3개 abstract public void ControlSwitch(); // 스위치 조작 추상 메소드.</pre>	기본적인 스위치 오브젝트의 상위 추상 클래스입니다. 스위치 조작 시 효과음 재생을 위해 sfxManager를 할당하고, 인터페이스를 상속받는 메소드 ControlObject()에 스위치 조작 메소드인 ControlSwitch() 추상 메소드를 실행할 수 있도록 합니다.
<pre>public abstract class OnOffSwitch : SwitchObject { private SpriteRenderer switchSpriteRenderer; // 스위 [SerializeField] private Sprite onSwitchSprite; // 켜진 [SerializeField] private Sprite offSwitchSprite; // 꺼진 @Unity 메시지 참조: 0개 private void Awake() { // 스위치 오브젝트의 스프라이트렌더러 가져옴. switchSpriteRenderer = GetComponent<SpriteRenderer>(); switchSpriteRenderer.sprite = offSwitchSprite; // 꺼진 } 참조: 2개 public override void ControlSwitch() { // 스위치 스프라이트가 꺼져있으면 켜고, if (switchSpriteRenderer.sprite == offSwitchSprite) { switchSpriteRenderer.sprite = onSwitchSprite; } else { // 스위치 스프라이트가 켜져있으면 끄. switchSpriteRenderer.sprite = offSwitchSprite; } ControlOnOffSwitch(); } 참조: 2개 abstract public void ControlOnOffSwitch(); // 스위치 켜고 끄</pre>	다음은 SwitchObject를 상속받는 OnOffSwitch 추상 클래스입니다. 이 클래스는 켜고 끄는 2가지 상태의 스위치를 위해 만들었습니다. 이 스크립트를 상속받는 오브젝트는 인스펙터에서 스위치가 켜진/꺼진 상태의 스프라이트를 넣어주어야 합니다. 그리고, ControlSwitch() 를 override하여 동작을 작성해줍니다. 스프라이트를 변경하고, ControlOnOffSwitch() 추상메소드를 통해 실질적인 동작을 실행하도록 작성했습니다.
<pre>// 문을 열고닫는 스위치 클래스. @Unity 스크립트 (자신 참조: 2개) 참조: 0개 public class OpenCloseDoorSwitch : OnOffSwitch { // 이 버튼이 열고 닫을 문 오브젝트들의 정보들과 메소드 들어있음. [SerializeField] private Door[] doors; public override void ControlOnOffSwitch() { OpenCloseDoor(); } // 문을 열고 닫는 메소드. 참조: 1개 public void OpenCloseDoor() { if (doors != null) { // 스위치가 열고 닫을 모든 문들을 확인하며, foreach (Door door in doors) { // 해당 문이 열려있으면, if (door.IsOpenedDoor() == true) { Debug.Log("문 닫음"); door.CloseDoor(); // 문을 닫음. } else { Debug.Log("문 열음"); door.OpenDoor(); // 아니면, 문을 열음. } } } } }</pre>	다음은 OnOffSwitch를 상속받는 OpenCloseDoorSwitch 클래스입니다. 이 클래스는 문을 열고 닫는 스위치를 위해 만들었습니다. 이 스크립트를 갖는 오브젝트는 ControlOnOffSwitch() 를 override 하여 OpenCloseDoor()를 실행시키고, OpenCloseDoor()는 Door[]에 들어있는 문들에 대해 열고 닫는 메소드를 실행함으로써 열린 문은 닫고, 닫힌 문은 열도록 하는 실질적인 동작을 수행하도록 하는 메소드입니다.
	스위치 오브젝트(OpenCloseDoorSwitch) 의 인스펙터입니다. 레버 이미지를 할당하고, 열거나 닫을 문을 넣어줍니다.  (직접 그린 스위치 이미지들)
	적용된 스위치 오브젝트(프리팸) 들

7-3. 버튼 오브젝트

```
// (조작가능한 오브젝트인 버튼)의 상위 추상 클래스. (이지만, IControlObject 상속 x => 상호작용을
// Unity 스크립트 | 참조: 1개
public abstract class ButtonObject : MonoBehaviour
{
    private SfxManager sfxManager;

    private LinkedList<GameObject> collidedPlayers; // 충돌한 플레이어들 리스트.
    private SpriteRenderer buttonSpriteRenderer; // 버튼 오브젝트의 스프라이트렌더러
    [SerializeField] private Sprite baseButtonSprite; // 기본 버튼 스프라이트.
    [SerializeField] private Sprite pushedButtonSprite; // 눌린 버튼 스프라이트.

    // 처음에는 충돌한 플레이어들 리스트를 초기화.
    // Unity 메시지 | 참조: 0개
    private void Awake()
    {
        sfxManager = GameManager.instance.gameObject.transform.GetChild(0).GetComponent<SfxManager>();

        collidedPlayers = new LinkedList<GameObject>();

        // 버튼 오브젝트의 스프라이트렌더러 가져옴.
        buttonSpriteRenderer = GetComponent<SpriteRenderer>();
        buttonSpriteRenderer.sprite = baseButtonSprite; // 기본 버튼 스프라이트로 초기화.
    }
}
```

기본적인 버튼 오브젝트의 상위 추상 클래스입니다.

버튼 오브젝트도 상호작용하는 오브젝트이지만, IControlObject 인터페이스를 상속하지 않습니다. 버튼 오브젝트는 상호작용 키를 눌러 상호작용하지 않고, 밟을 때 상호작용하기 때문입니다.

충돌 플레이어들의 리스트 변수도 만들었습니다. 이유는 한 버튼을 둘 이상의 플레이어가 밟을 때, 이미 밟힌 상태임에도 또 밟아서 중복으로 작동하는 것을 방지하기 위함입니다.

```
// collider(trigger)가 접촉되면,
// Unity 메시지 | 참조: 0개
private void OnTriggerEnter2D(Collider2D collision)
{
    // 충돌한 오브젝트의 태그가 플레이어이면, => 플레이어와 충돌 시.
    if (collision.CompareTag("Player") == true)
    {
        // 충돌한 플레이어들 리스트에 충돌한 해당 플레이어 추가.
        collidedPlayers.AddLast(collision.gameObject);
    }

    // 충돌 중인 플레이어가 있다면, 버튼을 누름.
    if (collidedPlayers.Count > 0)
    {
        sfxManager.PlayButtonSound();

        buttonSpriteRenderer.sprite = pushedButtonSprite;
        PushButton(); // 버튼 누름.
    }
}

// 플레이어 접촉 해제 시.
// Unity 메시지 | 참조: 0개
private void OnTriggerExit2D(Collider2D collision)
{
    // 충돌 해제된 오브젝트의 태그가 플레이어이면, => 플레이어와 충돌 해제
    if (collision.CompareTag("Player") == true)
    {
        // 충돌 해제된 플레이어가 리스트에 있으면, 리스트에서 지움.
        if (collidedPlayers.Contains(collision.gameObject))
        {
            collidedPlayers.Remove(collision.gameObject);
        }

        // 충돌 중인 플레이어가 없다면, 버튼을 땀.
        if (collidedPlayers.Count == 0)
        {
            buttonSpriteRenderer.sprite = baseButtonSprite;
            CancelButton(); // 버튼 누름 해제.
        }
    }
}
```

버튼을 충돌했을 때 작동하게 하기 위해서, OnTriggerEnter2D() 와 OnTriggerExit2D() 를 이용했습니다.

버튼이 플레이어와 충돌했을 때에는, 우선 리스트에 플레이어를 추가하고, 충돌한 플레이어가 있는 경우에만, 버튼을 누르도록 했습니다.

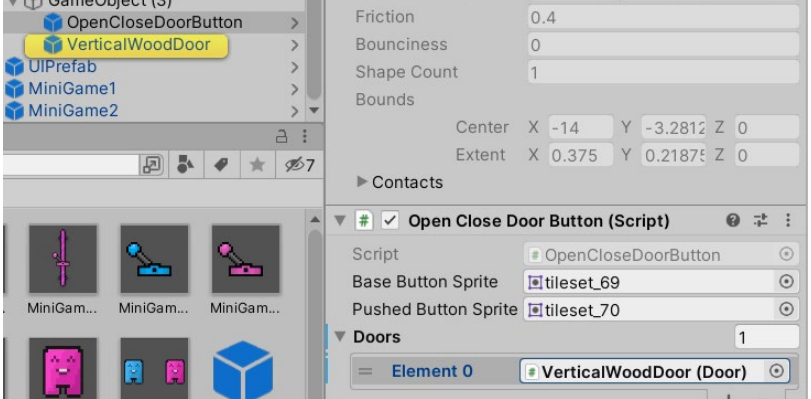
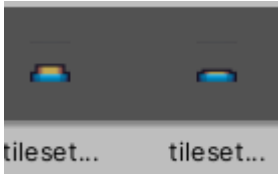
버튼에 충돌 해제될 때에는, 해당 플레이어를 리스트에서 지우고, 충돌한 플레이어가 없는 경우에만 버튼을 누르는 것을 해제하도록 했습니다.

```
// 버튼 누름.
// 참조: 2개
public abstract void PushButton();

// 버튼 누름 해제.
// 참조: 2개
public abstract void CancelButton();
```

버튼을 누르는 추상 메소드와 버튼 누름 해제를 하는 추상 메소드를 만들어 주었습니다. 이 추상 클래스를 상속받는 클래스에서 실질적인 동작을 수행합니다.

다음은 실질적인 동작을 수행하는 OpenCloseDoorButton 클래스입니다.

<pre>// 버튼 누름. // 구조 2개 public override void PushButton() { // 이 버튼이 눌리지 않았으면, if (isPushed == false) { Debug.Log("버튼을 누름."); // 버튼을 누름. isPushed = true; foreach (Door door in doors) { // 해당 문이 열려있으면, if (door.IsOpenedDoor() == true) { Debug.Log("문 닫음."); door.CloseDoor(); // 문을 닫음. } else { Debug.Log("문 열음."); door.OpenDoor(); // 아니면, 문을 열음. } } } } // 버튼 누름 해제. // 구조 2개 public override void CancelButton() { // 이 버튼이 눌리고 있으면, if (isPushed == true) { Debug.Log("버튼을 땀."); // 버튼을 땀. isPushed = false; foreach (Door door in doors) { // 해당 문이 열려있으면, if (door.IsOpenedDoor() == true) { Debug.Log("문 닫음."); door.CloseDoor(); // 문을 닫음. } else { Debug.Log("문 열음."); door.OpenDoor(); // 아니면, 문을 열음. } } } }</pre>	PushButton()과 CancelButton()을 Override 하여 실질적으로 연결된 문을 열고 닫는 메소드를 만들어 주었습니다.
	버튼 오브젝트(OpenCloseDoorDoor) 의 인스펙터입니다. 버튼 이미지를 할당하고, 열거나 닫을 문을 넣어줍니다.  (유니티 에셋으로 다운받은 버튼 이미지)
	적용된 버튼 오브젝트(프리팹) 들

7-4. 문 오브젝트

<pre>private SfxManager sfxManager; private Collider2D doorCollider2D; // 문 오브젝트 콜라이더. private SpriteRenderer doorSpriteRenderer; // 문 오브젝트의 스프라이트렌더러. [SerializeField] private Sprite openedDoorSprite; // 열린 문 스프라이트. [SerializeField] private Sprite closedDoorSprite; // 닫힌 문 스프라이트. [SerializeField] private bool isOpened; // 문이 열려있는지 저장하는 변수. 인스펙터에서 초기값 지정. private void Awake() { sfxManager = GameManager.instance.gameObject.transform.GetChild(0).GetComponent<SfxManager>(); // 오브젝트의 콜라이더와 스프라이트렌더러 가져옴. doorCollider2D = GetComponent<Collider2D>(); doorSpriteRenderer = GetComponent<SpriteRenderer>(); // 변수 초기값에 따라 문의 상태를 설정. if (isOpened == true) // 변수 상태가 true이면, 열어줌. { isOpened = true; doorCollider2D.enabled = false; // 지나갈 수 있도록 콜라이더 꺼주고, doorSpriteRenderer.sprite = openedDoorSprite; // 스프라이트를 열린 문 이미지로. } else { isOpened = false; doorCollider2D.enabled = true; // 지나갈 수 없도록 콜라이더 켜주고, doorSpriteRenderer.sprite = closedDoorSprite; // 스프라이트를 닫힌 문 이미지로. } }</pre>	기본적인 문 오브젝트의 클래스입니다. 초기화를 위해 isOpened 변수를 두어 초기에 열어 놓을 것인지 닫아 놓을 것인지 설정할 수 있게 했습니다. 문을 열고 닫는 것을 콜라이더를 끄고 켜는 것으로 구현했습니다. 이를 위해 콜라이더 변수의 enabled 를 true/false 로 해주었습니다.
---	---

```
// 문을 여는 메소드.
public void OpenDoor()
{
    sfxManager.PlayDoorOpenSound();

    isOpened = true;
    doorCollider2D.enabled = false; // 지나갈 수 있도록 콜라이더 꺼주고,
    doorSpriteRenderer.sprite = openedDoorSprite; // 스프라이트를 열린 문 이미지로.
}

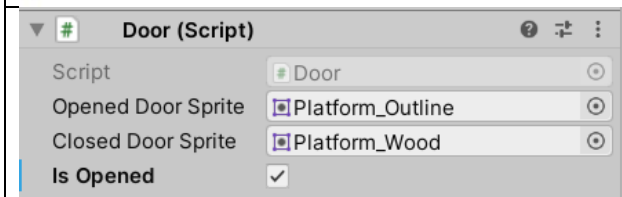
// 문을 닫는 메소드.
public void CloseDoor()
{
    sfxManager.PlayDoorCloseSound();

    isOpened = false;
    doorCollider2D.enabled = true; // 지나갈 수 없도록 콜라이더 켜주고,
    doorSpriteRenderer.sprite = closedDoorSprite; // 스프라이트를 닫힌 문 이미지로.
}

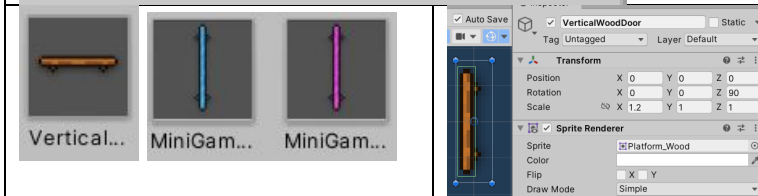
// 문이 열려있는지 확인하는 메소드.
public bool IsOpenedDoor()
{
    return (isOpened == true); // 열려있으면 true 반환.
}
```

OpenDoor() 메소드와 CloseDoor() 메소드는 문을 열고 닫는 메소드입니다. 각각의 효과음을 재생하고, isOpened 변수를 설정하고, 콜라이더를 끄거나 켜고, 스프라이트를 변경해줍니다.

IsOpenedDoor() 메소드는 해당 문이 열려있는지 닫혀있는지 반환하는 함수입니다.



Door 스크립트 컴포넌트 역시 열고 닫히기 때문에, 인스펙터에서 열린/닫힌 문 스프라이트를 넣어주고, 인스펙터에서 열렸는지 닫혔는지 초기값을 설정할 수도 있습니다.



적용된 문 오브젝트(프리팸) 들이 때, 나무 문은 원래 스프라이트가 플랫폼이라서 z축 90도 회전시켜 표현했습니다.



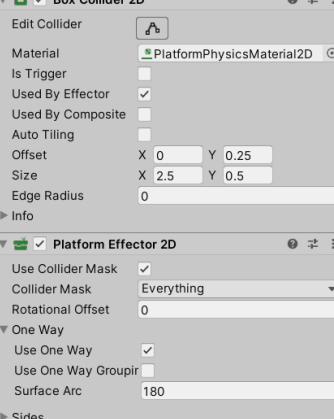
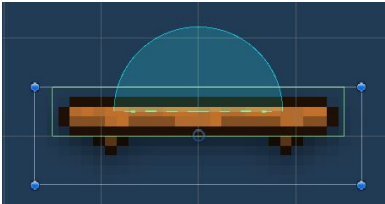
(유니티 에셋으로 다운받은 문(플랫폼) 이미지)
아래의 플랫폼(발판)에도 사용하였습니다.

7-5. 플랫폼 오브젝트

플랫폼 오브젝트는 문과 거의 동일한 오브젝트입니다. 따라서 Door.cs 스크립트 컴포넌트를 공유하지만, 문 오브젝트와 다른 점이 하나 있습니다.



문 오브젝트는 닫힌 경우, 양방향 모두 이동이 불가능 하지만,
플랫폼 오브젝트는 닫힌 경우, 아래에서 위로만 갈 수 있지만, 위에서 아래로 갈 수는 없어야 합니다.

		이를 구현해주기 위해서, platform Effector 2D 컴포넌트를 추가하고 Use One Way에 체크해줍니다. 다음, 콜라이더에서 Used By Effector를 체크해줍니다. 적용이 끝나면, 적용된 오브젝트에 다음과 같은 푸른 반투명한 반원이 생깁니다.
		적용된 문 오브젝트(프리팸) 들이 때, 나무 문은 원래 스프라이트가 플랫폼이라서 z축 90도 회전시켜 표현했습니다.
		(유니티 에셋으로 다운받은 문(플랫폼) 이미지) 아래의 플랫폼(발판)에도 사용하였습니다.

8. 퍼즐들

직소 퍼즐 관련 클래스와 메소드는

“유니티 – 직소퍼즐구현1 (<https://game-happy-world.tistory.com/43>) 를 참고하여 구현하였습니다

우선, 3가지 퍼즐들의 동일한 기능을 위해 퍼즐들에 대해서 추상 클래스를 만들어 주었습니다.

8-1. Puzzle 클래스는 다음에 소개할 FruitPuzzle, LineMatchingPuzzle, PasswordPuzzle 가 Puzzle 추상 클래스를 상속합니다.

```
public abstract class Puzzle : MonoBehaviour
{
    protected SfxManager sfxManager;

    public bool IsClear { get; private set; }
    public void SetClear(bool value) { IsClear = value; }

    public GameObject PuzzlePosSet { get; protected set; }
    public GameObject PuzzlePieceSet { get; protected set; }
    public GameObject PuzzleMovingSet { get; protected set; }

    참조 10개
    public int[] DefaultPiecesPosSerial { get; protected set; }
    참조 8개
    public int[] DefaultPosSerial { get; protected set; }

    @ Unity 메시지 | 참조 0개
    private void Start()
    {
        sfxManager = GameManager.instance.gameObject.transform.

// 주어진 배열의 길이만큼 0~n-1의 번호를 부여하고 랜덤으로 섞
참조 4개
public void RandomizeSerial(int[] serial)
{
    int len = serial.Length;
    for (int i = 0; i < len; i++)
    {
        serial[i] = i;
    }

    for (int i = 0; i < len; i++)
    {
        // 무작위 a, b 값을 가져옴. [0 ~ len-1]
        int a = UnityEngine.Random.Range(0, len);
        int b = UnityEngine.Random.Range(0, len);

        // 튜플을 이용하여 값 교환.
        (serial[b], serial[a]) = (serial[a], serial[b]);
    }

// 퍼즐 조각을 맞추는 영역의 위치를 번호에 맞게 초기화 하는 !
참조 5개
public abstract void InitPosSet();

// 퍼즐이 완성되었는지 확인하는 메소드.
참조 7개
public abstract void CheckClear();
}
```

<Puzzle 추상 클래스>

이 추상클래스에는, 클리어 여부를 위한 IsClear변수와 SetClear() 메소드가 있고,

조각 퍼즐들을 위한, PuzzlePosSet, PuzzlePieceSet, PuzzleMovingSet 게임오브젝트가 있습니다.

그리고, 조각을 섞는 것을 도와줄 시리얼 배열인 DefaultPiecePosSerial과 DefaultPosSerial이 있습니다.

RandomizeSerial() 메소드는 시리얼을 조각 개수만큼 0부터n-1까지 시리얼을 만들어 이를 튜플에 이용하여 값을 교환하여 섞어주는 메소드입니다.

InitPosSet() 메소드는 퍼즐 조각이 들어가는 영역의 위치를 시리얼의 번호에 맞게 위치를 초기화해주는 추상 메소드 입니다.

CheckClear() 메소드는 퍼즐이 완성되었는지 체크하는 추상 메소드입니다.

다음은 퍼즐 조각에 대해 동일한 기능을 수행하는 것을 묶어주기 위해서 인터페이스를 만들었습니다.

8-2. IPuzzlePiece 인터페이스.

```
public interface IPuzzlePiece : IDragHandler, IEndDragHandler, IBeginDragHandler
{
    참조 9개
    public int PieceNum { get; set; }

    // 퍼즐 조각의 위치를 번호에 맞게 초기화 하는 메소드.
    참조 4개
    public void InitPosPiece();

    // 퍼즐 조각을 맞출 때(스냅할 때), 해당 위치에 이미 조각이 있는지 확인해서 스냅하는 메소드.
    참조 4개
    public bool CheckSnapPuzzle();
}
```

<IPuzzlePiece 인터페이스>

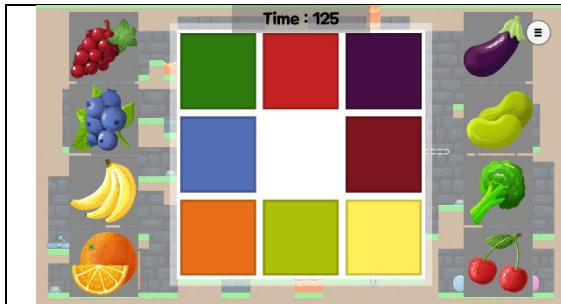
퍼즐 조각은 마우스로 조작해야 하기 때문에, IDragHandler, IEndDragHandler, IBeginDragHandler 인터페이스를 추가해줍니다. 각각 마우스 드래그 중일 때, 마우스 드래그가 끝날 때, 마우스 드래그가 시작할 때를 위한 인터페이스들입니다.

PieceNum 프로퍼티는 각 조각의 번호를 위한 프로퍼티입니다.

InitPosPiece() 메소드는 각 퍼즐 조각의 위치를 시리얼 번호에 맞게 초기화하는 메소드입니다.

CheckSnapPuzzle() 메소드는 퍼즐 조각을 맞출 때, 해당 위치에 이미 조각이 있는지 확인하여 없으면 스냅하고, 그렇지 않으면(이미 조각이 있거나, 스냅거리가 조각과 영역의 거리보다 먼 경우) 스냅하지 않는 메소드입니다.

8-3. 퍼즐 1) 과일 퍼즐 – 직소 퍼즐을 참고한 퍼즐로, 모든 퍼즐 조각을 색깔을 맞추어 배치하면 클리어 됩니다.



<구현 화면>

```
public class FruitsPuzzle : Puzzle
{
    [Unity 메시지 | 참조 0개]
    private void Awake()
    {
        PuzzlePosSet = transform.GetChild(0).gameObject;
        PuzzlePieceSet = transform.GetChild(1).gameObject;
        PuzzleMovingSet = transform.GetChild(2).gameObject;

        DefaultPiecesPosSerial = new int[8];
        DefaultPosSerial = new int[8];

        RandomizeSerial(DefaultPiecesPosSerial);
        RandomizeSerial(DefaultPosSerial);

        InitPosSet();
    }
}
```

과일 퍼즐은 FruitPuzzle.cs 스크립트로 작성하였고, 이는 Puzzle 추상 클래스를 상속받기 때문에, PuzzlePosSet, PuzzlePieceSet, PuzzleMovingSet을 초기화해주고, DefaultPiecePosSerial과 DefaultPosSerial 배열을 new로 할당하고, RandomizeSerial() 메소드를 이용하여 시리얼을 섞어주어 조각과 영역의 초기값을 랜덤화해줍니다. 이 덕분에, 퍼즐을 실행할 때마다 조각과 영역의 위치가 랜덤이 되어, 다양한 가짓수를 가질 수 있도록 하였습니다.

```
// 퍼즐 조각을 맞추는 영역의 위치를 번호에 맞게 초기화 하는 메소드.
참조 2개
public override void InitPosSet()
{
    Vector2[] defaultPuzzlePos = { new Vector2(-300, 300), new Vector2(0, 300), new Vector2(300, 300),
        new Vector2(300, 0), new Vector2(300, -300), new Vector2(0, -300), new Vector2(-300, -300), new Vector2(-300, 0) };

    // 퍼즐 조각 위치 초기화.
    for (int i = 0; i < PuzzlePosSet.transform.childCount; i++)
    {
        PuzzlePosSet.transform.GetChild(i).GetComponent<RectTransform>().localPosition = defaultPuzzlePos[DefaultPosSerial[i]];
    }
}

참조 3개
public override void CheckClear()
{
    for (int i = 0; i < PuzzlePosSet.transform.childCount; i++)
    {
        //퍼즐위치의 자식이 없으면 모든 퍼즐조각이 놓여지지 않은것입니다.
        if (PuzzlePosSet.transform.GetChild(i).childCount == 0)
        {
            return;
        }
        //퍼즐조각의 번호와 퍼즐 위치 번호가 일치하지 않으면 퍼즐은 완성되지 않은것입니다.
        if (PuzzlePosSet.transform.GetChild(i).GetChild(0).GetComponent<IPuzzlePiece>().PieceNum != i)
        {
            return;
        }
    }

    sfxManager.PlayPuzzleClearSound();

    SetClear(true);
}
}
```

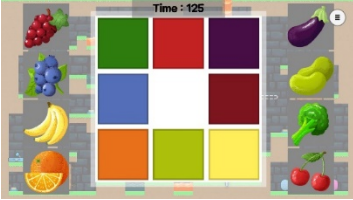
InitPosSet() 메소드를 override하여 작성해주었고, 과일 퍼즐의 기본 위치를 지정해주었고, 이를 영역의 시리얼에 맞게 초기화해주었습니다. CheckClear() 메소드를 override하여 작성해주었고, 모든 퍼즐의 조각이 맞는 경우가 아니면, return을 통해 클리어시키지 않고, 모든 퍼즐의 조각이 맞는 경우에만, 퍼즐 클리어 효과음을 재생하고, SetClear(true)를 통해 클리어했다고 알립니다.

다음은 과일 퍼즐의 퍼즐 조각을 위한, FruitPuzzlePiece.cs 스크립트입니다.

```
// 퍼즐 조각의 위치를 번호에 맞게 초기화 하는 메소드.
참조 2개
public void InitPosPiece()
{
    // 퍼즐 조각 위치 초기화.
    int defaultPosValue = puzzle.DefaultPiecesPosSerial[PieceNum];
    int posX, posY;

    posX = (defaultPosValue - (puzzle.DefaultPiecesPosSerial.Length / 2) >= 0 ? 715 : -715);
    posY = defaultPosValue % (puzzle.DefaultPiecesPosSerial.Length / 2);
    posY = (posY * 260) - 390;

    GetComponent<RectTransform>().localPosition = new Vector2(posX, posY);
}
```



InitPos()에서 위 그림과 같이 양 옆 4개씩의 위치를 초기화해줍니다.

```
// 드래그 시작.
참조 0개
public void OnBeginDrag(PointerEventData eventData)
{
    sfxManager.PlayPuzzlePieceDragSound();
    // 지금 드래그하는 퍼즐 조각이 가장 위로 올 수 있도록 처리
    transform.SetParent(puzzleMovingSet.transform);
}

// 드래그 중.
참조 0개
public void OnDrag(PointerEventData eventData)
{
    // 마우스 커서 위치 따라다니도록 처리.
    transform.position = eventData.position;
}

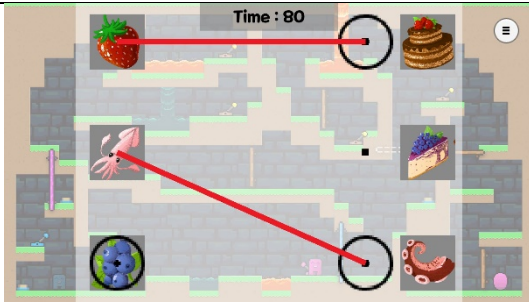
// 드래그 종료.
참조 0개
public void OnEndDrag(PointerEventData eventData)
{
    if (!CheckSnapPuzzle())
    {
        sfxManager.PlayPuzzlePieceDropSound();
        // 퍼즐 조각을 스냅시키려는 위치에 조각이 있거나 퍼즐
        transform.SetParent(puzzlePieceSet.transform);
    }
    else
    {
        sfxManager.PlayPuzzlePieceSnapSound();
        // 퍼즐 완성 확인.
        puzzle.CheckClear();
    }
}
```

IDragHandler, IBeginDragHandler, IEndDragHandler 에 의해서, OnDrag(), OnBeginDrag(), OnEndDrag()를 오버라이딩하여 작성해야 하고, 각각의 처리를 해주었습니다. 이때, 드래그 중에 시각적으로 다른 조각에 가리지 않기 위해서 MovingSet 오브젝트를 추가하고, SetParent() 메소드를 이용해, 레이어 상에 위쪽으로 올 수 있도록 처리했습니다.

```
public bool CheckSnapPuzzle()
{
    for (int i = 0; i < puzzlePosSet.transform.childCount; i++)
    {
        // 위치에 자식오브젝트가 있으면 이미 퍼즐조각이 놓여진 것입니다.
        if (puzzlePosSet.transform.GetChild(i).childCount != 0)
        {
            continue;
        }
        // 자식 오브젝트가 없는 퍼즐pos의 위치를 확인하며 스냅오프셋보다 적으면, 스냅. (중심으로부터 snap 거리 이내이면, 스냅)
        else if (Vector2.Distance(puzzlePosSet.transform.GetChild(i).position, transform.position) < snapOffset)
        {
            transform.SetParent(puzzlePosSet.transform.GetChild(i).transform);
            transform.localPosition = Vector3.zero;
            return true;
        }
    }
    return false;
}
```

CheckSnapPuzzle() 역시 PuzzlePiece 인터페이스를 오버라이딩하여 작성했고, 상기의 블로그를 참고하여 작성했습니다.

8-4. 퍼즐 2) 선 연결 퍼즐 – 선 연결 퍼즐을 참고한 퍼즐로, 모든 퍼즐 조각을 알맞은 것끼리 연결하면 클리어 됩니다.



<구현 화면>

선 연결 퍼즐의 구현은 직소퍼즐을 응용하여 만들었습니다. 이미지를 부모로 만들어 놓고, 타겟이 되는 검은 원이 조각, 목표 영역이 되는 검은 네모가 영역으로 보면 같다고 볼 수 있습니다.

스냅의 기준이 타겟이 되는 원이기 때문에, 검은 사각형이 원 내부에 있다면 스냅이 되도록 했습니다.

스냅이 되지 않으면 다시 원위치로 돌아가도록 했습니다.

또한, 빨간 선을 추가해서 늘어나도록 했습니다.

```
// 드래그 시작.
참조 0개
public void OnBeginDrag(PointerEventData eventData)
{
    sfxManager.PlayPuzzlePieceDragSound();
    // 지금 드래그하는 퍼즐 조각이 가장 위로 올 수 있도록 처리.
    transform.SetParent(puzzleMovingSet.transform);
}

// 드래그 중.
참조 0개
public void OnDrag(PointerEventData eventData)
{
    // 마우스 커서 위치 따라다니도록 처리.
    transform.position = eventData.position;
    // 빨간 줄도 오브젝트의 방향과 거리에 맞게 늘려줌.
    line.MatchLine();
}

// 드래그 종료.
참조 0개
public void OnEndDrag(PointerEventData eventData)
{
    if (!CheckSnapPuzzle())
    {
        sfxManager.PlayPuzzlePieceDropSound();
        // 퍼즐 조각을 스냅시키려는 위치에 조각이 있거나 퍼즐을 맞추는 위치가 아니면, 스냅
        transform.SetParent(puzzlePieceSet.transform.GetChild(PieceNum).transform);
        // 원래 자리로 이동.
        myRectTransform.localPosition = new Vector2(0, 0);
        // 빨간 줄도 오브젝트의 방향과 거리에 맞게 늘려줌.
        line.MatchLine();
    }
    else
    {
        sfxManager.PlayPuzzlePieceSnapSound();
        // 빨간 줄도 오브젝트의 방향과 거리에 맞게 늘려줌.
        line.MatchLine();
        // 퍼즐 완성 확인.
        puzzle.CheckClear();
    }
}
```

LineMatching line;

IDragHandler, IBeginDragHandler, IEndDragHandler 에 의해서, OnDrag(), OnBeginDrag(), OnEndDrag()를 오버라이딩하여 작성해야 하고, 각각의 처리를 해주었습니다. 이때, 드래그 중에 시각적으로 다른 조각에 가리지 않기 위해서 MovingSet 오브젝트를 추가하고, SetParent() 메소드를 이용해, 레이어 상에 위쪽으로 올 수 있도록 처리했습니다.

추가로 LineMatching 이라는 선을 연결하기 위한 변수를 추가하고, line.MatchLine() 메소드를 통해 빨간 선을 초기 위치와 조각 사이를 연결해주도록 했습니다.

나머지 부분은 과일퍼즐과 거의 유사한 것을 볼 수 있습니다.

선 연결을 위한 부분은 유니티 2D 마우스 드래그 라인 만들기 (<https://blog.naver.com/dooya-log/221431250246>) 참고하여 구현했습니다. <LineMatching.cs>

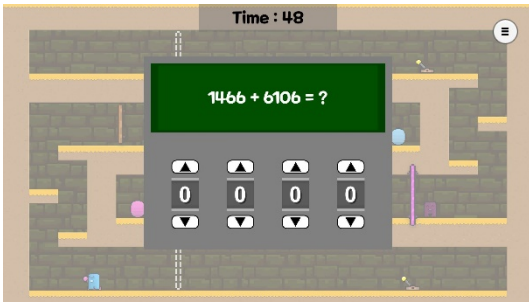
```
public void MatchLine()
{
    // 기준 오브젝트의 위치를 가져옴.
    originPos = originObj.GetComponent<RectTransform>().position;
    // 퍼즐 조각 오브젝트의 위치를 가져옴.
    targetPos = targetObj.GetComponent<RectTransform>().position;

    // 기준 오브젝트와 퍼즐 조각의 위치를 통해 두 사이의 거리를 계산.
    float distance = Vector2.Distance(originPos, targetPos);

    // 줄의 너비를 두 사이의 거리만큼 늘려주고,
    myRectTransform.sizeDelta = new Vector2(distance, myRectTransform.rect.height);
    // 기준 오브젝트와 퍼즐 조각의 각도만큼 회전시켜줌.
    myRectTransform.transform.localRotation = Quaternion.Euler(0, 0, AngleInDeg(originPos, targetPos));
    // => 두 오브젝트 사이에 선이 연결된 것처럼 보임.
}
```

회전은 localRotation(), 길이(너비)는 sizeDelta를 이용했습니다.

8-5. 퍼즐 3) 비밀번호 퍼즐 – 방 탈출 게임에서 자주 사용되는 퍼즐을 참고한 퍼즐로, 문제에 대한 답을 맞히면 클리어 됩니다.



비밀번호 퍼즐은 앞의 두 퍼즐과는 다르게 퍼즐 조각이 없는 퍼즐이기 때문에, IPuzzlePiece를 상속받는 부분이 없습니다. 퍼즐의 주요 부분만 설명하겠습니다.

```
// 텍스트 초기화 메소드.
참조 1개
private void InitTexts()
{
    questionText.text = MakeQuestion(); // 문제는 만들어준 값으로 초기화하고,

    for (int i = 0; i < size; i++) // 슬롯은 '0' 으로 초기화.
    {
        slotTexts[i].text = "0";
    }
}

// 문제 만드는 메소드.
참조 1개
private string MakeQuestion()
{
    int questionValue = UnityEngine.Random.Range(0, (int)Mathf.Pow(10, size));

    answer = questionValue.ToString().PadLeft(size, '0'); // 부족한 부분은 0으로

    int a = UnityEngine.Random.Range(0, questionValue); // a와 b 두 부분으로
    int b = questionValue - a; // a + b = answer 가

    return $"{a} + {b} = ?"; // "a + b = ?" 형식의 문자열을 만들어 리턴.
}
```

MakeQuestion() 메소드를 통해 문제를 랜덤으로 생성해내고,

```
참조 1개
public void UpNumber()
{
    sfxManager = GameManager.instance.gameObject.transform.GetChild(0).GetComponent<SfxManager>();
    sfxManager.PlayButtonSound();

    GameObject go = EventSystem.current.currentSelectedGameObject; // EventSystem.current.currentSelectedGameObject

    Text t = go.transform.parent.GetChild(2).GetChild(0).GetComponent<Text>(); // 현재 선택된 버튼의 텍스트
    int currentNum = int.Parse(t.text); // 텍스트의 정수문자를 정수로 변환.
    currentNum = (currentNum + 1) % 10; // 현재 정수값에 1을 더해주고 10으로 mod 연산. (9 -> 0 -> 1)

    t.text = currentNum.ToString(); // 1을 더해준 값을 다시 문자열로 변환하여 슬롯의 텍스트로 설정.

    CheckClear(); // 정답인지 확인.
}

// 해당 슬롯의 숫자를 하나 내리는 메소드.
참조 1개
public void DownNumber()
{
    sfxManager = GameManager.instance.gameObject.transform.GetChild(0).GetComponent<SfxManager>();
    sfxManager.PlayButtonSound();

    GameObject go = EventSystem.current.currentSelectedGameObject; // EventSystem.current.currentSelectedGameObject

    Text t = go.transform.parent.GetChild(2).GetChild(0).GetComponent<Text>();
    int currentNum = int.Parse(t.text);
    currentNum = (currentNum - 1 + 10) % 10; // 현재 정수값에 1을 빼주고 10으로 mod 연산. (1 -> 0 -> 9)

    t.text = currentNum.ToString();

    CheckClear();
}
```

UpNumber(), DownNumber() 메소드를 버튼에 연결하여 값을 입력하도록 합니다.

이 때, EventSystem.current.currentSelectedGmeObject 를 통해 현재 선택된 버튼을 가져올 수 있게 했습니다.

```
public override void InitPosSet()
{
    // 조각이 없는 퍼즐이라 위치 초기화 필요 없음.
    // 따라서, overriding하여 내용을 구현하지는 않는다
}

참조 5개
public override void CheckClear()
{
    if (answer.CompareTo(GetSlotNumbers()) == 0) /
    {
        sfxManager.PlayPuzzleClearSound();
        SetClear(true);
    }
}
```

Puzzle을 상속받기 때문에, 두 메소드를 override 해주었습니다.

9. 씬 이동 관련

```
public class SceneController
{
    // 현재 씬을 다시 시작하는 메소드.
    참조 37회
    public static void Restart()
    {
        Debug.Log("Restart");
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
        GameManager.instance.GameResume();
        GameManager.instance.Init();
    }

    // 시작 화면으로 가는 메소드.
    참조 47회
    public static void GotoHome()
    {
        Debug.Log("GotoHome");
        SceneManager.LoadScene("Start");
        GameManager.instance.GameResume();
        GameManager.instance.Init();
    }
}

// 다음 스테이지 레벨 씬으로 가는 메소드. (Game1 -> Game2)
참조 17회
public static void GotoNext()
{
    Debug.Log("GotoNext");
    string sceneName = SceneManager.GetActiveScene().name;
    string nStr = sceneName.Substring(sceneName.Length - 1);
    int n = int.Parse(nStr);
    // 최대 스테이지 이전 스테이지이면, 다음 스테이지로 이동.
    if (1 <= n && n < GameManager.instance.LevelCount)
    {
        string nextName = $"Game{n + 1}";
        SceneManager.LoadScene(nextName);
        GameManager.instance.GameResume();
        GameManager.instance.Init();
    }
    else // 만약 마지막 스테이지이면, 시작 화면으로 이동.
    {
        GotoHome();
    }
}
```

SceneController.cs 스크립트를 작성하였는데, MonoBehaviour를 상속받지 않고, static 메소드를 이용하여 구현했습니다.

단순히 SceneController.Restart(), SceneController.GotoHome(), SceneController.GotoNext()로 사용이 가능하도록 했습니다.

Restart() 메소드는 LoadScene(SceneManager.GetActiveScene().name) 을 통해 현재 씬을 다시 로드하도록 했습니다.

GotoHome() 메소드는 LoadScene("Start") 를 통해 간단하게 구현했습니다.

GotoNext() 메소드는 현재 씬의 이름을 가져와서, 몇 번째 스테이지인지 확인하고, 마지막 스테이지가 아니면 다음 스테이지로 이동시키고, 마지막 스테이지이면, Start 씬으로 이동하도록 구현했습니다.

● 깃허브와 Github Desktop을 이용해 관리를 하였습니다.

(<https://github.com/Kim-WanKyu/TwoBlobsEscape>)

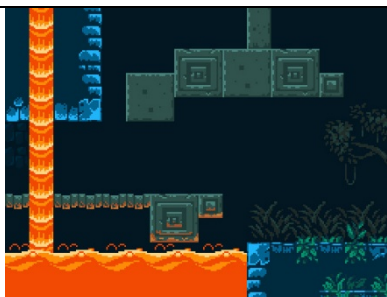
(<https://github.com/Kim-WanKyu/TwoBlobsEscape/commits/main/>)

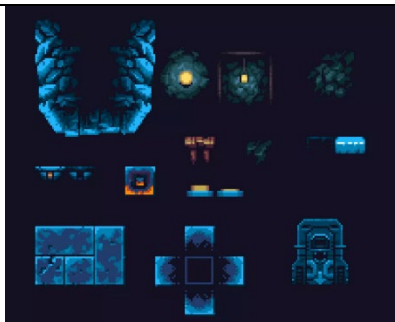
사용 에셋 출처

유니티 에셋

에셋명	Free Platform Game Assets	
제작자	BayatGames	
링크	https://assetstore.unity.com/packages/2d/envir onments/free-platform-game-assets-85838	
사용처	시작 씬 배경과 지형에, 배경, 지형 이미지(2D Spring/Autumn 128x128) 이미지를 사용했습니다.	

에셋명	Free 2D Mega Pack	
제작자	Brackeys	
링크	https://assetstore.unity.com/packages/2d/free-2d-mega-pack-177430	
사용처	게임 씬 배경과, 플레이어, 문, 플랫폼에 배경, 캐릭터, 플랫폼 이미지를 사용했습니다.	

에셋명	Grotto Escape II	
제작자	Ansimuz	
링크	https://assetstore.unity.com/packages/2d/textures-materials/grotto-escape-ii-86689	
사용처	물과 용암에 물과 용암 이미지들을 사용했습니다.	

에셋명	Super Grotto Escape Pack	
제작자	Ansimuz	
링크	https://assetstore.unity.com/packages/2d/environments/super-grotto-escape-pack-238393	
사용처	버튼 오브젝트에 버튼 이미지를 사용했습니다.	

에셋명	Food icons Pack	
제작자	Angelina Avgustova	
링크	https://assetstore.unity.com/packages/2d/gui/icons/food-icons-pack-70018	
사용처	퍼즐 조각에 과일과 오징어 이미지를 사용했습니다.	

사운드(SFX)

(일본어 무료 효과음 사이트)

効果音ラボ(효과음 연구실) (<https://soundeffect-lab.info/agreement/>)

[플레이어 관련]

じゅうたんの上を歩く (플레이어 걷기)

["じゅうたんの上を歩く"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

決定ボタンを押す39 (플레이어 점프)

["決定ボタンを押す39"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

水がジュースと蒸発 (플레이어 죽음)

["水がジュースと蒸発"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

水滴1 (플레이어 스왑 (교체))

["水滴1"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

[오브젝트 관련]

カーソル移動2 (버튼 오브젝트)

["カーソル移動2"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

カーソル移動8 (스위치 오브젝트)

["カーソル移動8"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

コンセントを挿す (문 열림)

["コンセントを挿す"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

ヤカンのふたを閉める (문 닫힘)

["ヤカンのふたを閉める"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

[장면 관련]

ゲージ回復3 (스테이지 시작)

["ゲージ回復3"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

自転車のベル (스테이지 클리어)

["自転車のベル"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

[퍼즐 관련]

自転車のベル (퍼즐 클리어)

["自転車のベル"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

決定ボタンを押す32 (퍼즐 조각 드래그 (퍼즐 조각을 들어올릴 때))

["決定ボタンを押す32"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

握手 (퍼즐 조각 드랍 (퍼즐 조각을 내려놓을 때))

["握手"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

雪玉をぶつける (퍼즐 조각 스냅 (퍼즐 조각을 영역에 맞출 때))

["雪玉をぶつける"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

[UI 관련]

カーソル移動9 (UI 버튼 클릭)

["カーソル移動9"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

パパッ (UI 창 띄움(켜기))

["パパッ"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)

ニュッ3 (UI 창 내림(끄기))

["ニュッ3"で検索した結果\[1\] | 効果音ラボ \(soundeffect-lab.info\)](#)